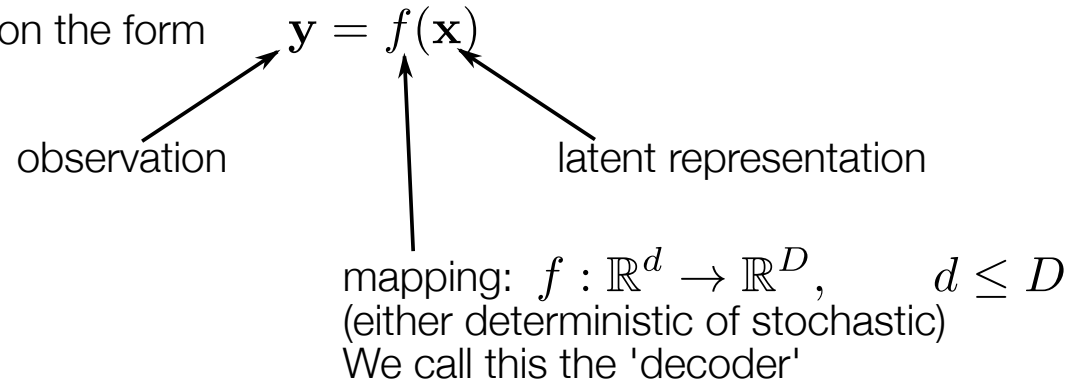


Week 7!

information geometry &
general parameters

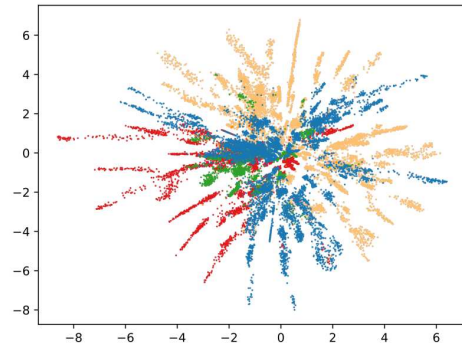
Recap of last week

We consider latent variable models on the form

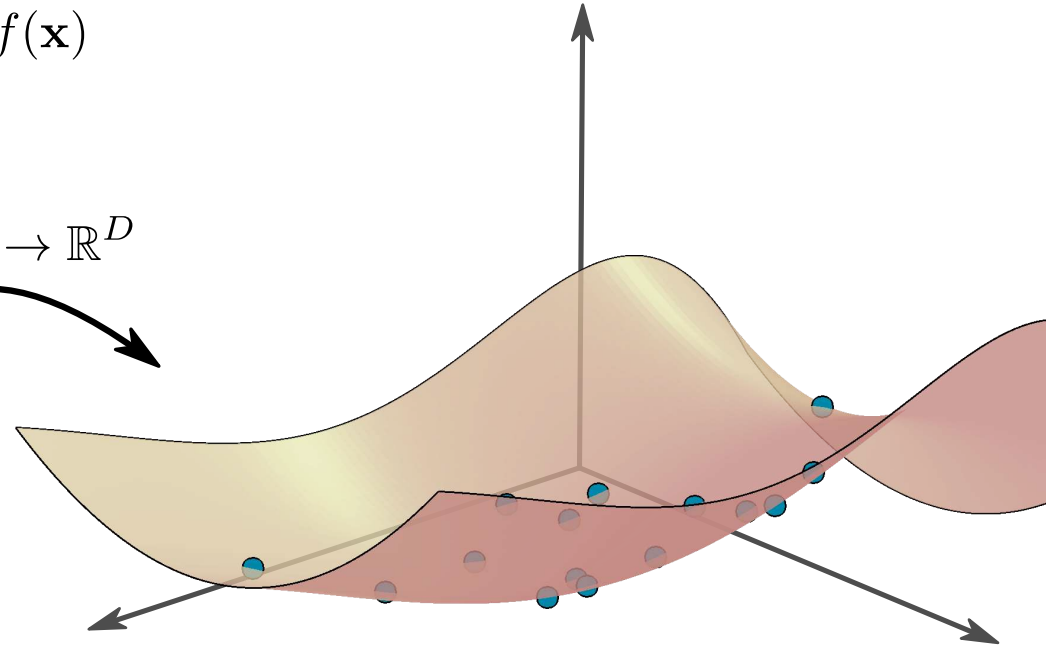


Recap of last week

We consider latent variable models on the form $\mathbf{y} = f(\mathbf{x})$

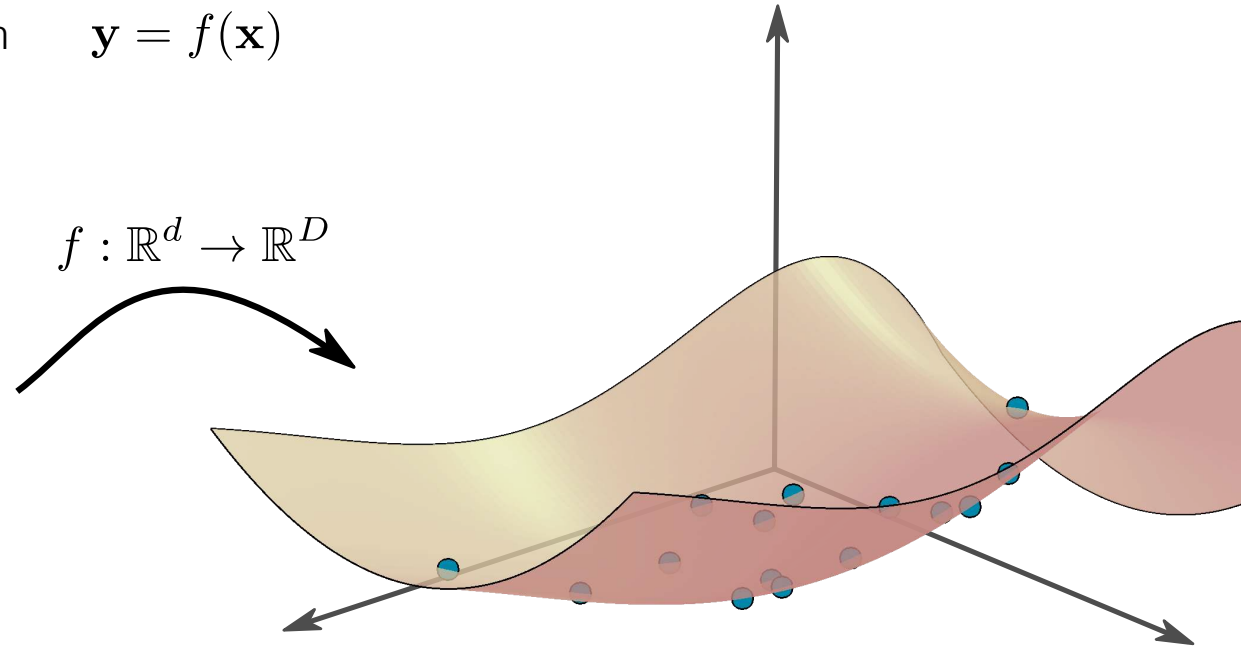
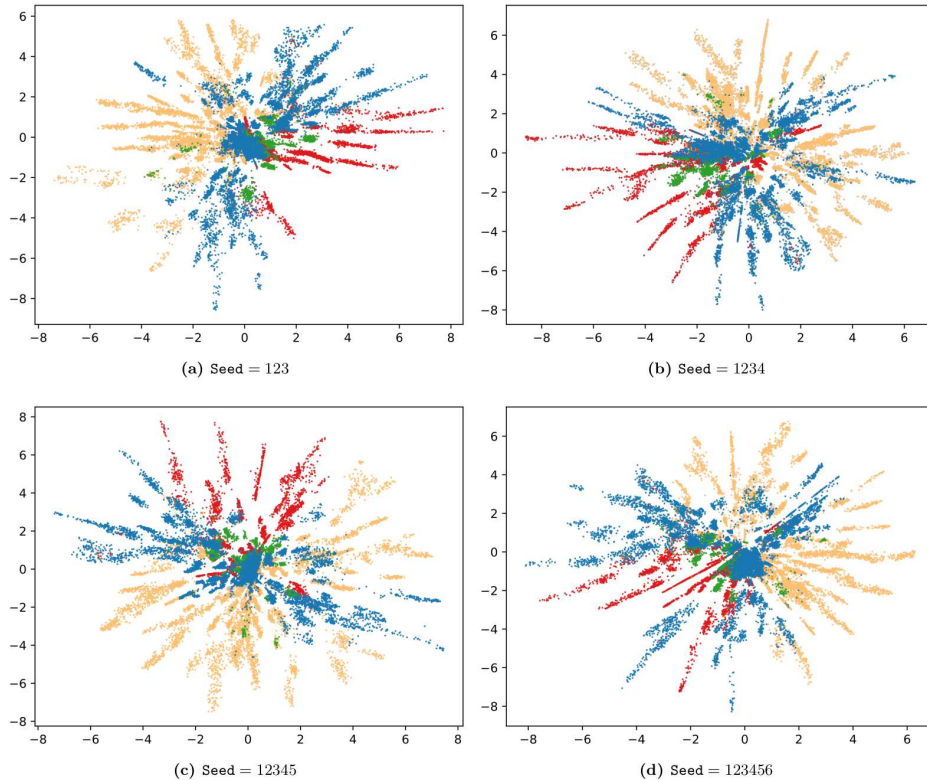


$$f : \mathbb{R}^d \rightarrow \mathbb{R}^D$$



Recap of last week

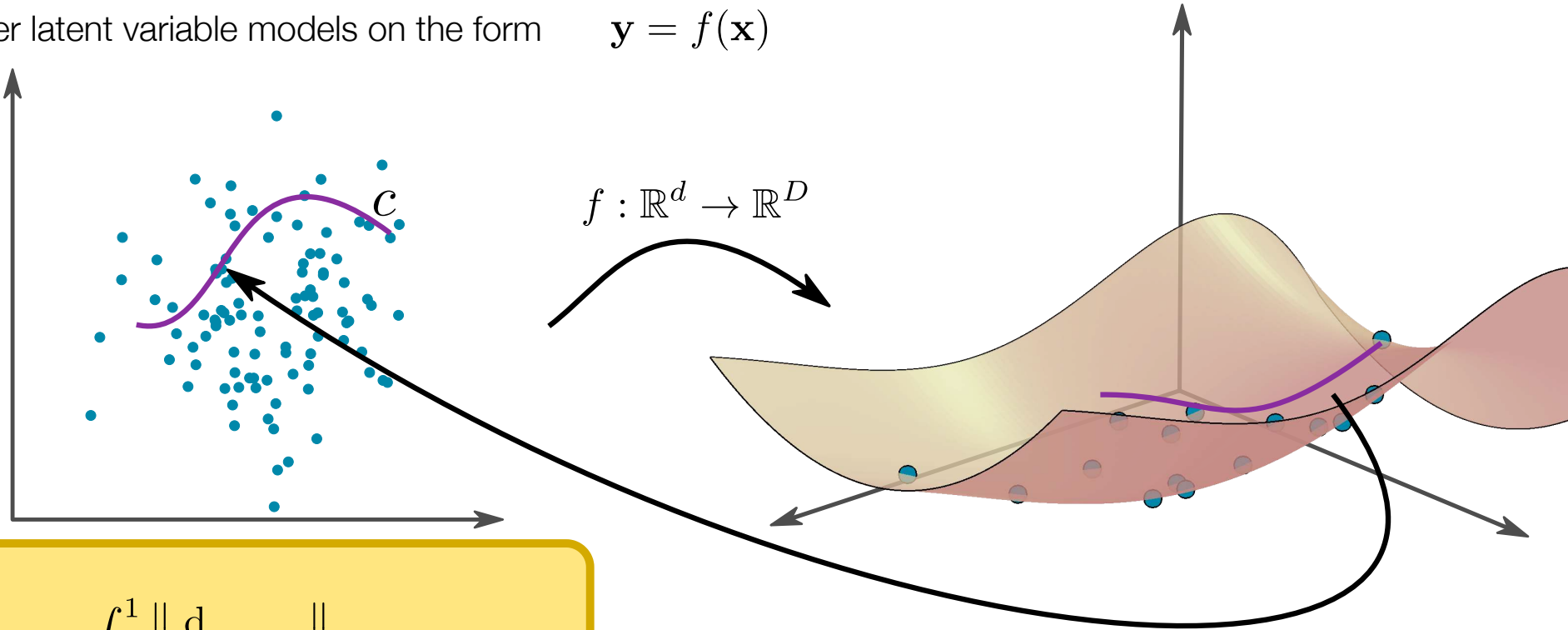
We consider latent variable models on the form $\mathbf{y} = f(\mathbf{x})$



Too much flexibility in the model (e.g. when using neural nets) allow us to reparametrize the latent representation without changing model fit. Renders latent representations unstable and unsuitable for posthoc processing/interpretation.

Recap of last week

We consider latent variable models on the form $\mathbf{y} = f(\mathbf{x})$



Define

$$\text{Length}(c) = \int_0^1 \left\| \frac{d}{dt} f(c_t) \right\| dt$$

The shortest path and associated distance is then

$$c_{01} = \underset{c}{\operatorname{argmin}} \text{Length}(f(c))$$

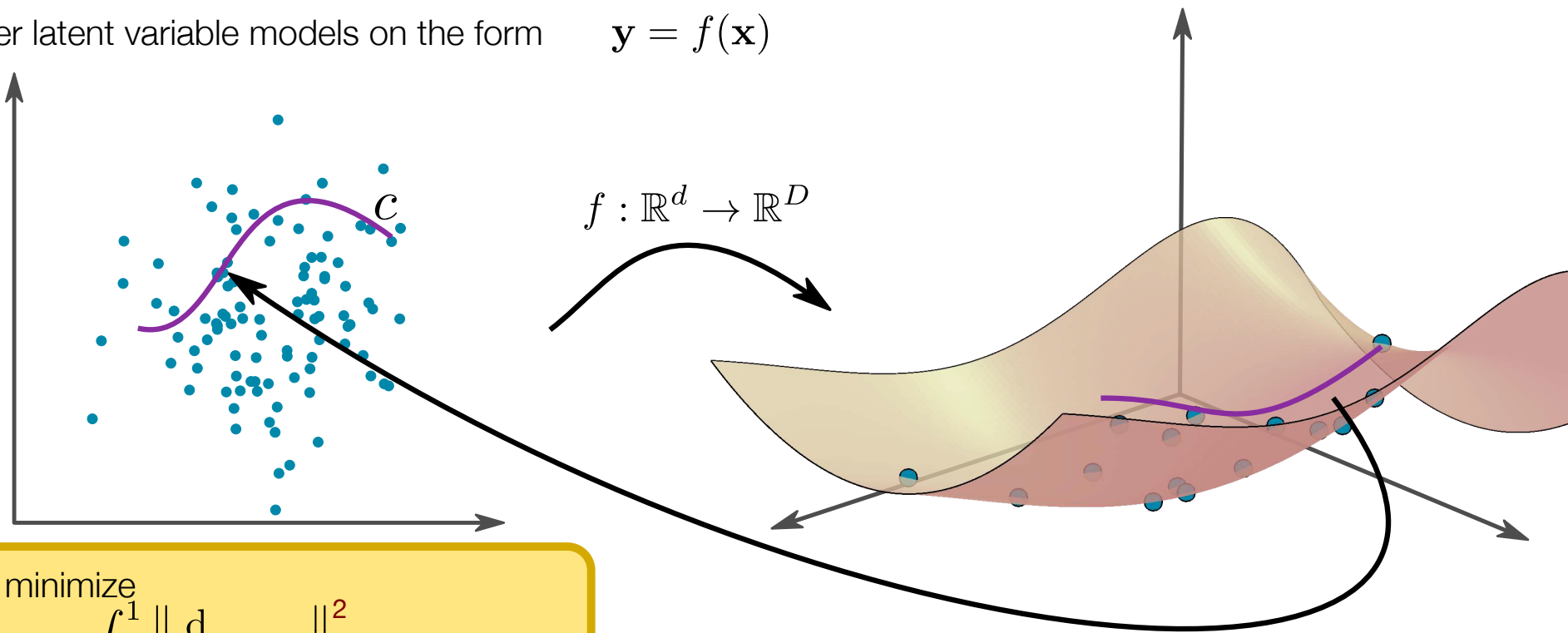
$$\text{dist}(c_0, c_1) = \text{Length}(c_{01})$$

Define length of *latent curve* to be the length of the *decoded curve*.

From that definition, you just go where the math takes you...

Recap of last week

We consider latent variable models on the form $\mathbf{y} = f(\mathbf{x})$



Numerically, we minimize

$$\text{Energy}(c) = \int_0^1 \left\| \frac{d}{dt} f(c_t) \right\|^2 dt$$

The shortest path and associated distance is then

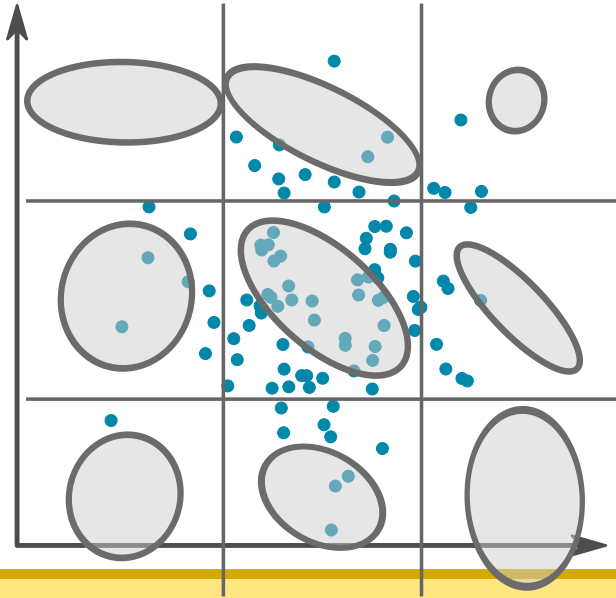
$$c_{01} = \underset{c}{\operatorname{argmin}} \text{Energy}(f(c))$$

$$\text{dist}^2(c_0, c_1) = \text{Energy}(c_{01})$$

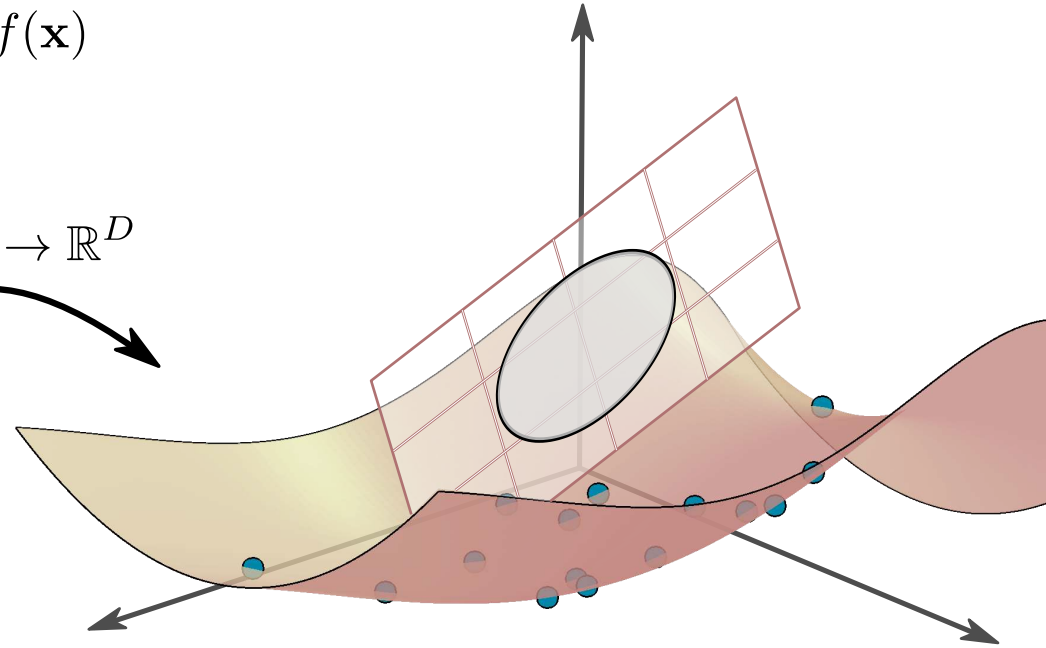
Define length of *latent* curve to be the length of the *decoded* curve.
From that definition, you just go where the math takes you...

Recap of last week

We consider latent variable models on the form $\mathbf{y} = f(\mathbf{x})$



$$f : \mathbb{R}^d \rightarrow \mathbb{R}^D$$



Locally, we can measure inner products

$$\langle \delta_1, \delta_2 \rangle_{\mathbf{x}} = (\mathbf{J}_{\mathbf{x}} \delta_1)^\top (\mathbf{J}_{\mathbf{x}} \delta_2) = \delta_1^\top \boxed{\mathbf{J}_{\mathbf{x}}^\top \mathbf{J}_{\mathbf{x}}} \delta_2$$

We call $\boxed{\mathbf{G}_{\mathbf{x}} = \mathbf{J}_{\mathbf{x}}^\top \mathbf{J}_{\mathbf{x}}}$ the (Riemannian) metric

Locally, we can measure inner products in the tangent space of our manifold. This gives rise to a local inner product in the latent space: a Riemannian metric.

I'm always looking for feedback



I need two volunteer "feedbackers"

(Gaussian)
Noisy manifold

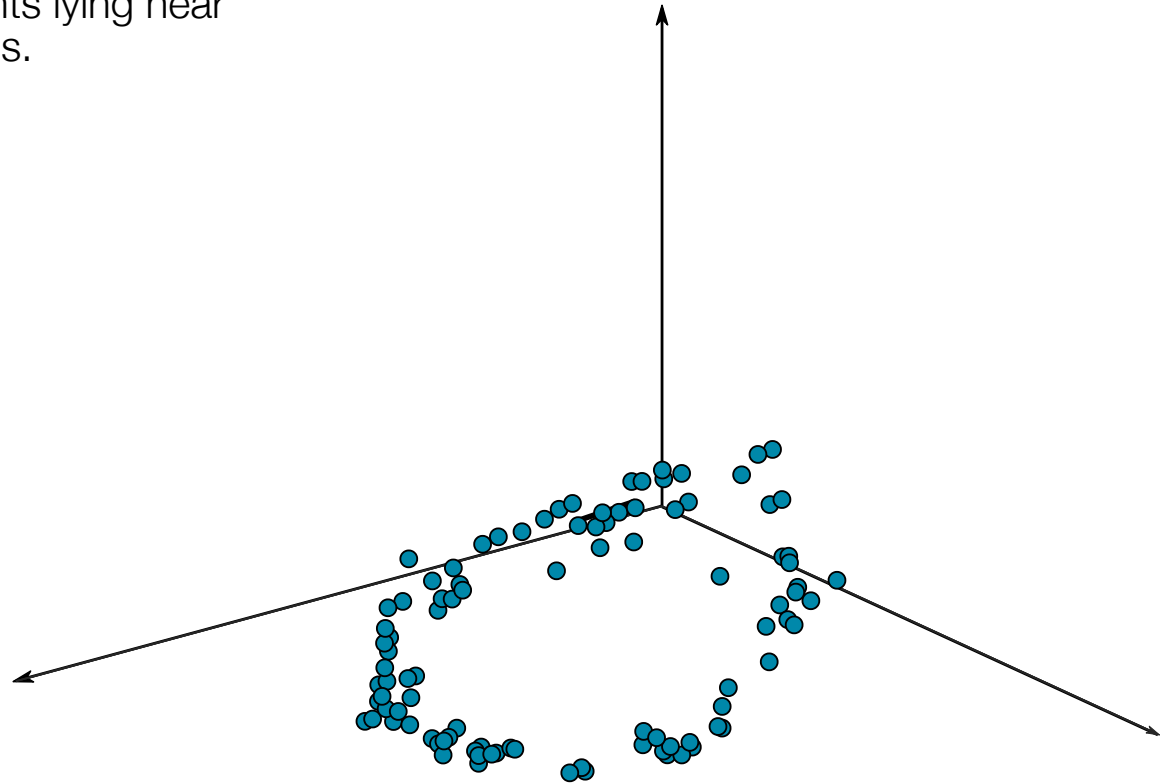
In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage with learned representations in models with a deterministic decoder.
Let's have a look at how it works in practice...

In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage with learned representations in models with a deterministic decoder. Let's have a look at how it works in practice...

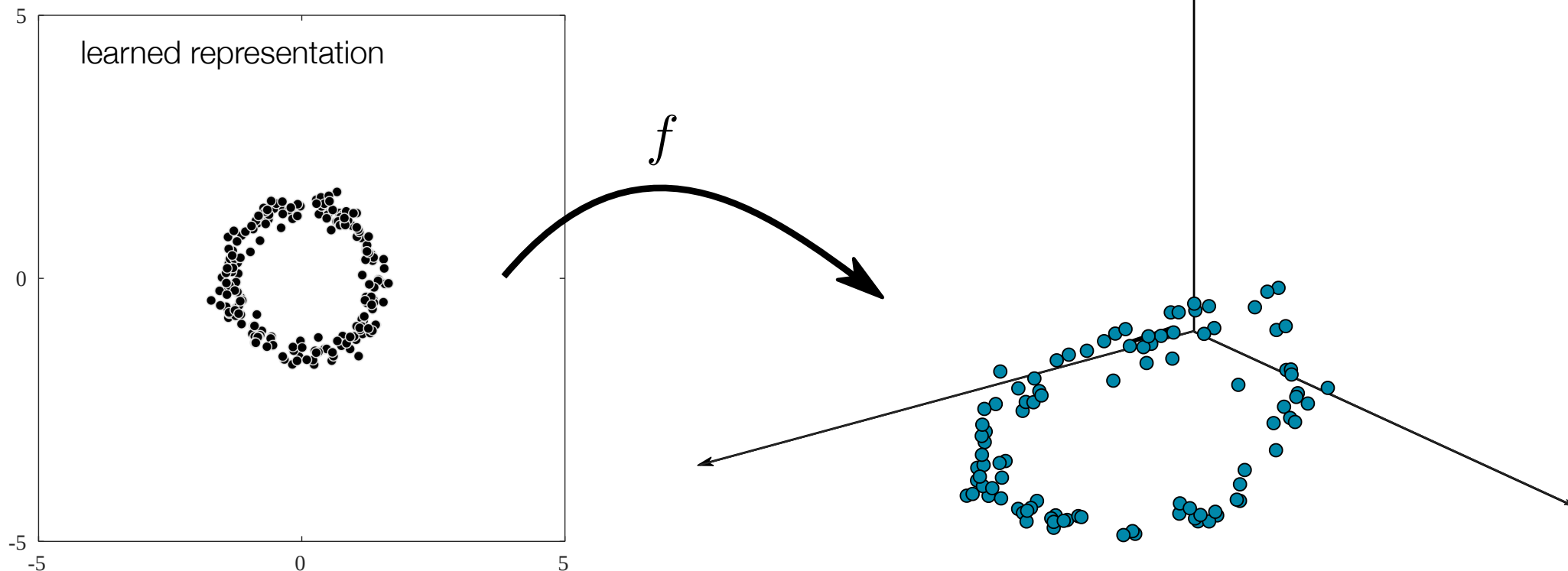
As a test, we construct a simple dataset of points lying near a deformed circle embedded in 1000 dimensions.



In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage with learned representations in models with a deterministic decoder. Let's have a look at how it works in practice...

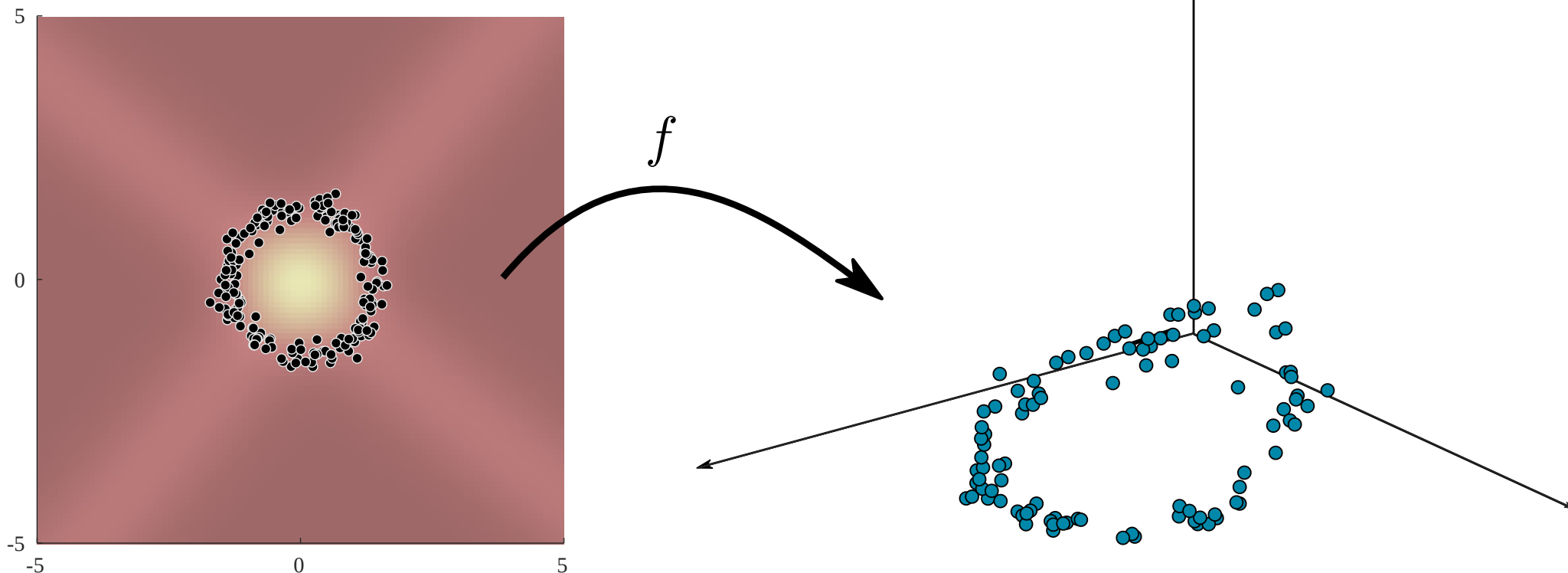
As a test, we construct a simple dataset of points lying near a deformed circle embedded in 1000 dimensions.



In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage with learned representations in models with a deterministic decoder. Let's have a look at how it works in practice...

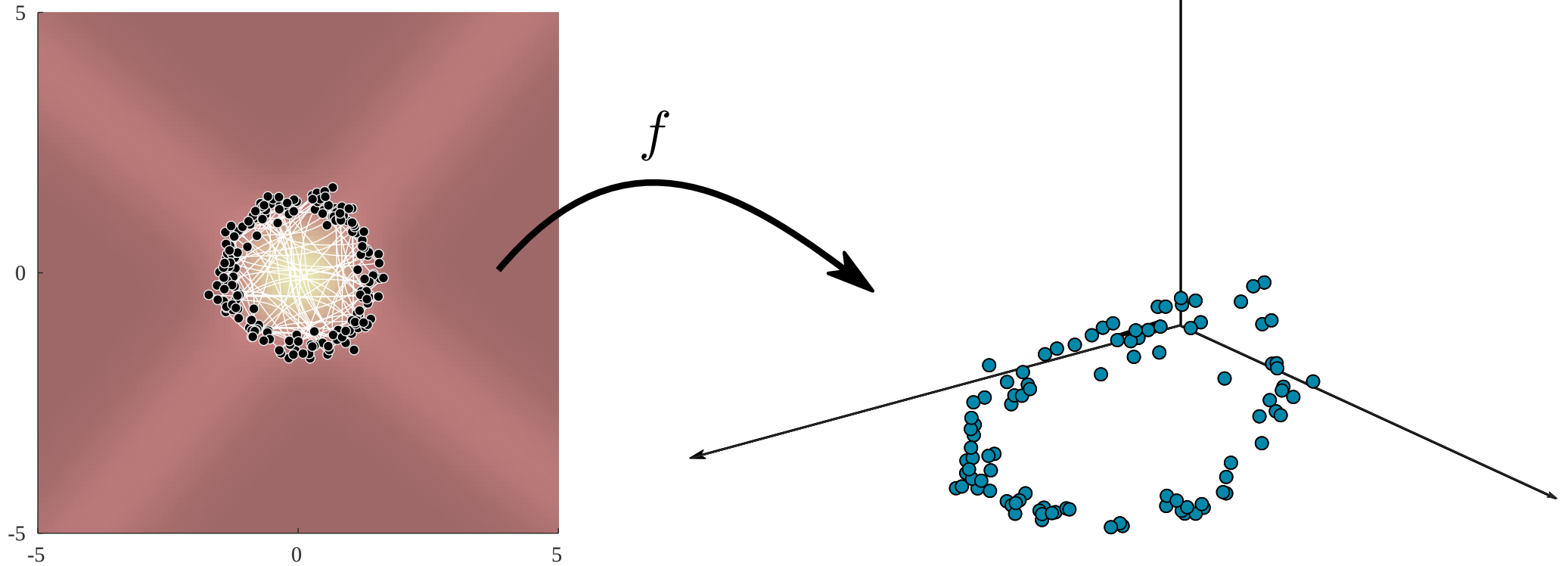
As a test, we construct a simple dataset of points lying near a deformed circle embedded in 1000 dimensions.



In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage with learned representations in models with a deterministic decoder. Let's have a look at how it works in practice...

As a test, we construct a simple dataset of points lying near a deformed circle embedded in 1000 dimensions.



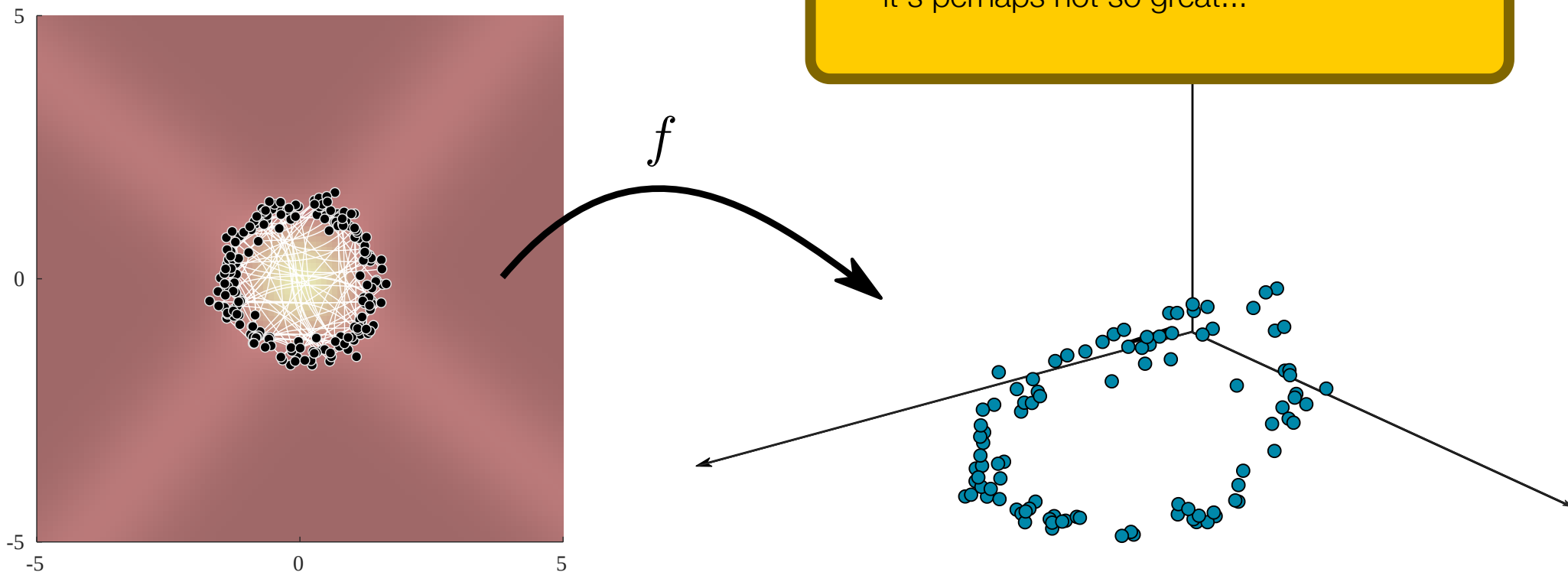
In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage learned representations in models with a deterministic distance. Let's have a look at how it works in practice...

As a test, we construct a simple dataset of points lying on a deformed circle embedded in 1000 dimensions.

So, geodesic distances are identifiable, but the underlying curves do not seem to represent what's actually happening in the data.

So, the theory is great, but in practice, it's perhaps not so great...



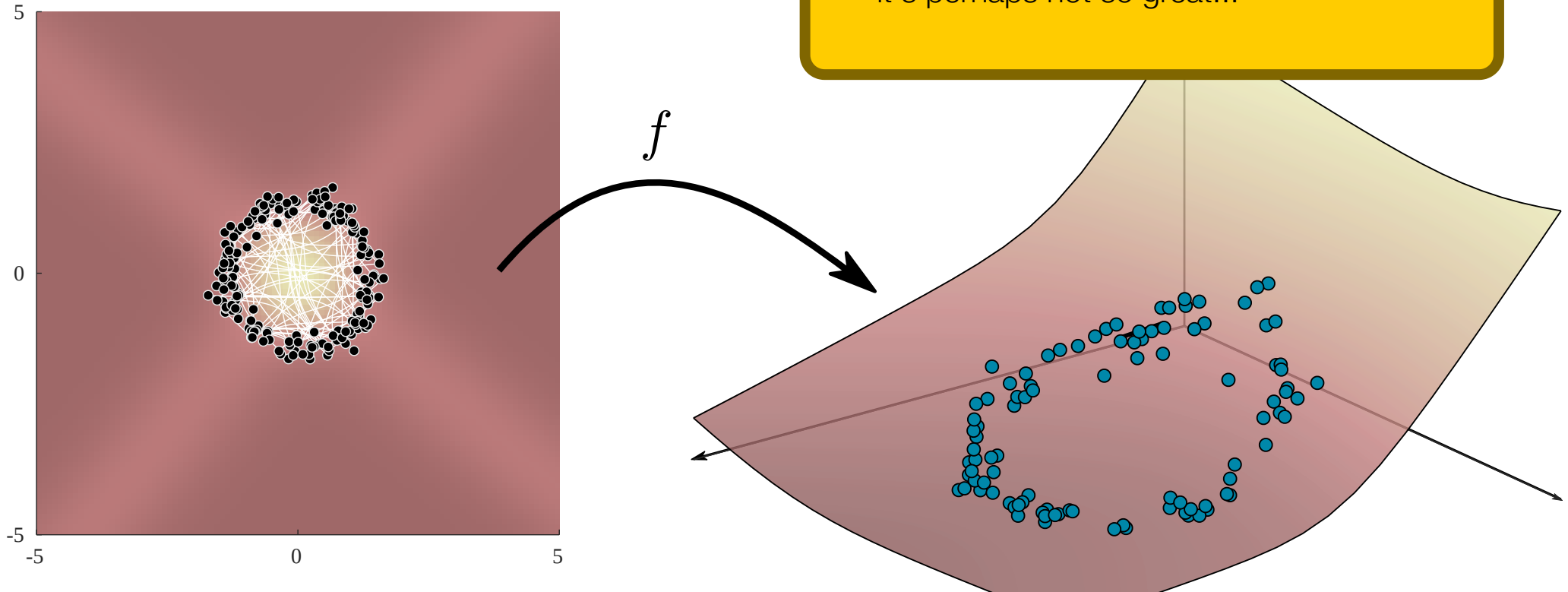
In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage learned representations in models with a deterministic distance. Let's have a look at how it works in practice...

As a test, we construct a simple dataset of points lying on a deformed circle embedded in 1000 dimensions.

So, geodesic distances are identifiable, but the underlying curves do not seem to represent what's actually happening in the data.

So, the theory is great, but in practice, it's perhaps not so great...



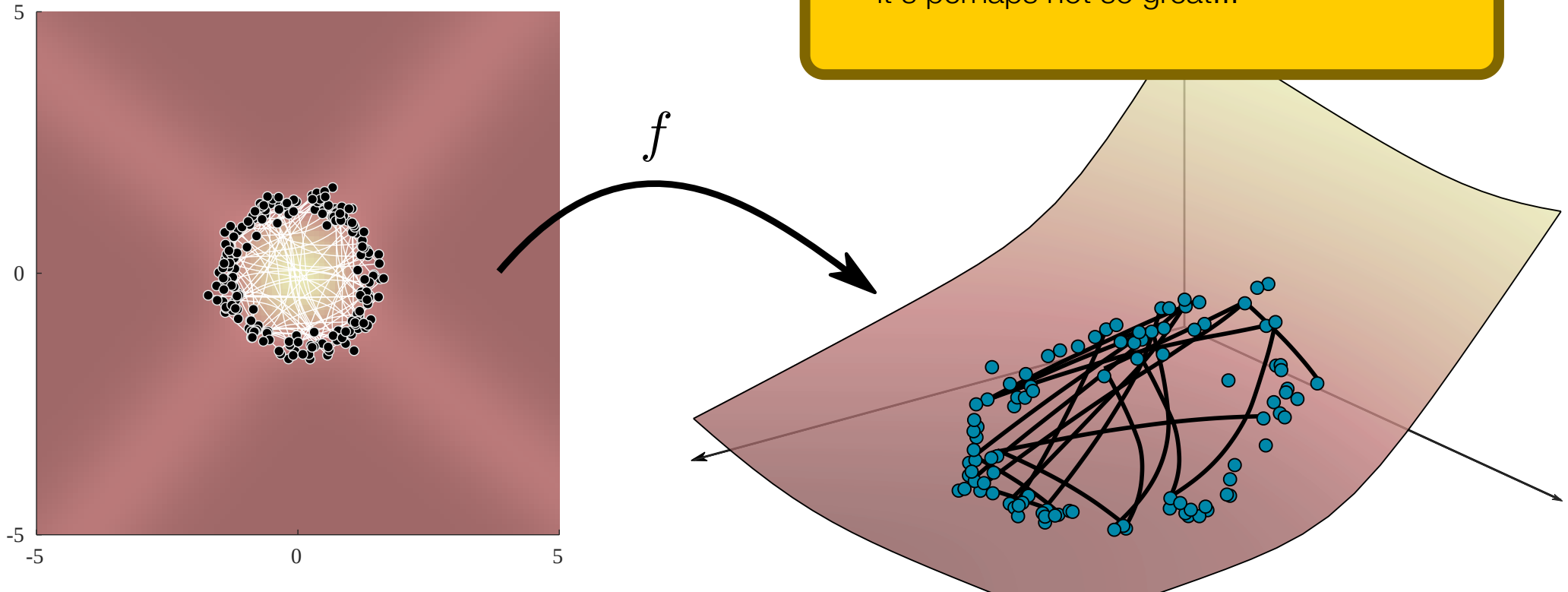
In theory, the theory should have worked; in practice...

We have constructed a set of tools that allow us to engage learned representations in models with a deterministic distance. Let's have a look at how it works in practice...

As a test, we construct a simple dataset of points lying on a deformed circle embedded in 1000 dimensions.

So, geodesic distances are identifiable, but the underlying curves do not seem to represent what's actually happening in the data.

So, the theory is great, but in practice, it's perhaps not so great...



Gaussian variational autoencoders

Consider the variational autoencoder with Gaussian output. This has a stochastic decoder of the form

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

where $\epsilon \in \mathbb{R}^{D \times D}$ is a diagonal random matrix with standard Gaussian entries $[\epsilon]_{ii} \sim \mathcal{N}(0, 1)$. and $\sigma(\mathbf{x}) \in \mathbb{R}^D$ is a smooth function that changes the scale of the additive noise.

Gaussian variational autoencoders

Consider the variational autoencoder with Gaussian output. This has a stochastic decoder of the form

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

where $\epsilon \in \mathbb{R}^{D \times D}$ is a diagonal random matrix with standard Gaussian entries $[\epsilon]_{ii} \sim \mathcal{N}(0, 1)$. and $\sigma(\mathbf{x}) \in \mathbb{R}^D$ is a smooth function that changes the scale of the additive noise.

It's a manifold plus noise

Gaussian variational autoencoders

Consider the variational autoencoder with Gaussian output. This has a stochastic decoder of the form

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

where $\epsilon \in \mathbb{R}^{D \times D}$ is a diagonal random matrix with standard Gaussian entries $[\epsilon]_{ii} \sim \mathcal{N}(0, 1)$. and $\sigma(\mathbf{x}) \in \mathbb{R}^D$ is a smooth function that changes the scale of the additive noise.

It's a manifold plus noise

But how do we deal with this noise geometrically?

We can choose to ignore (focusing on the mean), but then we are back to the models we just saw failing...

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P}h(\mathbf{x}).$$

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements.

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P}h(\mathbf{x}).$$

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements. We can interpret this as forming a random projection from $\mathbb{R}^{2D}$ to $\mathbb{R}^D$.

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P}h(\mathbf{x}).$$

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements. We can interpret this as forming a random projection from $\mathbb{R}^{2D}$ to $\mathbb{R}^D$.



Why is that the least bit interesting?

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P} h(\mathbf{x}).$$

We can think of $h(\mathbf{x})$ as spanning a d -dimensional manifold in $\mathbb{R}^{2D}$

The generative process is then a random projection of this manifold.

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements. We can interpret this as forming a random projection from $\mathbb{R}^{2D}$ to $\mathbb{R}^D$.



Why is that the least bit interesting?

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P} h(\mathbf{x}).$$

We can think of $h(\mathbf{x})$ as spanning a d -dimensional manifold in $\mathbb{R}^{2D}$.

The generative process is then a random projection of this manifold.

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements. We can interpret this as forming a random projection from $\mathbb{R}^{2D}$ to $\mathbb{R}^D$.



Why is that the least bit interesting?

The corresponding Riemannian metric is

$$\mathbf{G}_{\mathbf{x}} = \mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x}) + \mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x}).$$

Randomly projected manifolds

Let us rewrite the noisy manifold

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

into

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P} h(\mathbf{x}).$$

We can think of $h(\mathbf{x})$ as spanning a d -dimensional manifold in $\mathbb{R}^{2D}$.

The generative process is then a random projection of this manifold.

Here $\mathbf{P} = (\mathbf{I}, \epsilon) \in \mathbb{R}^{D \times 2D}$ is a random matrix with some deterministic elements. We can interpret this as forming a random projection from $\mathbb{R}^{2D}$ to $\mathbb{R}^D$.



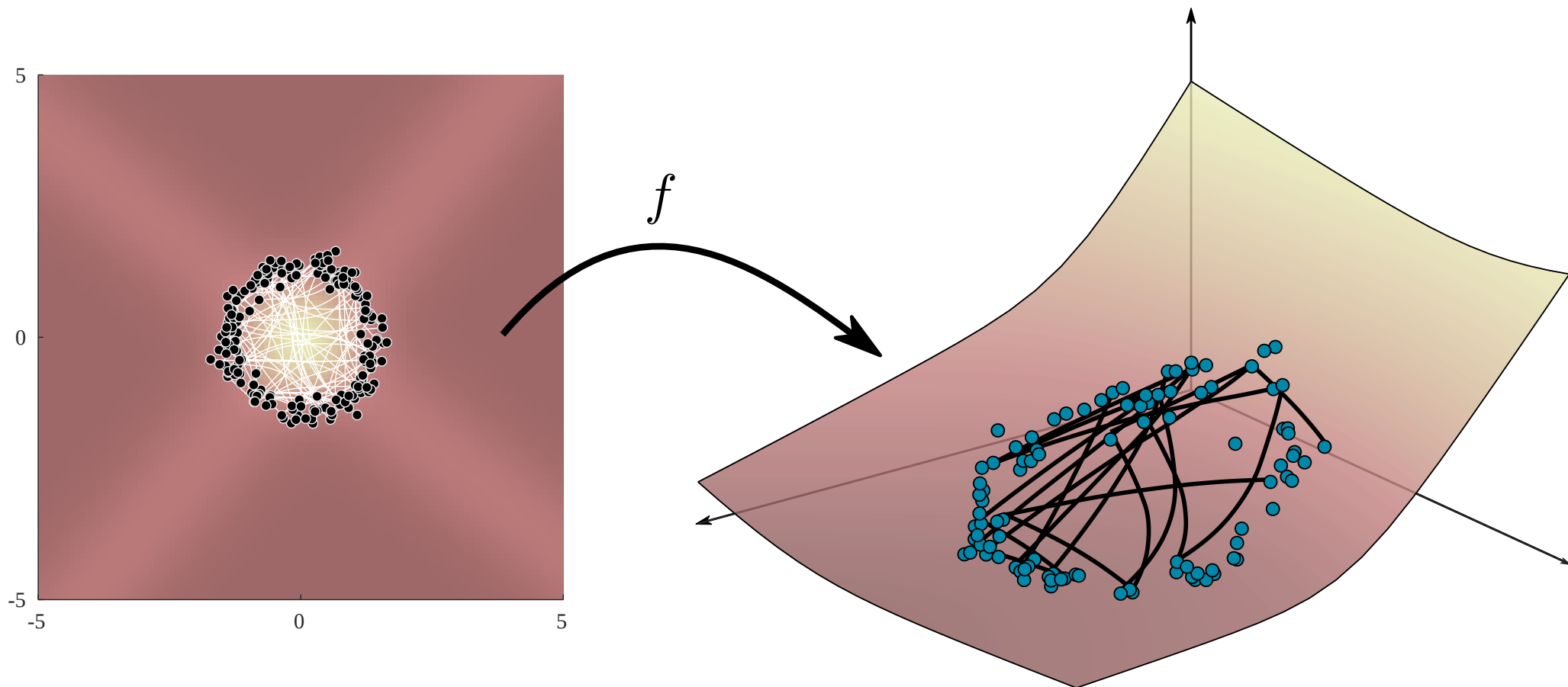
Why is that the least bit interesting?

The corresponding Riemannian metric is

$$\mathbf{G}_{\mathbf{x}} = \underbrace{\mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x})}_{\text{Mean term}} + \underbrace{\mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x})}_{\text{Uncertainty term}}.$$

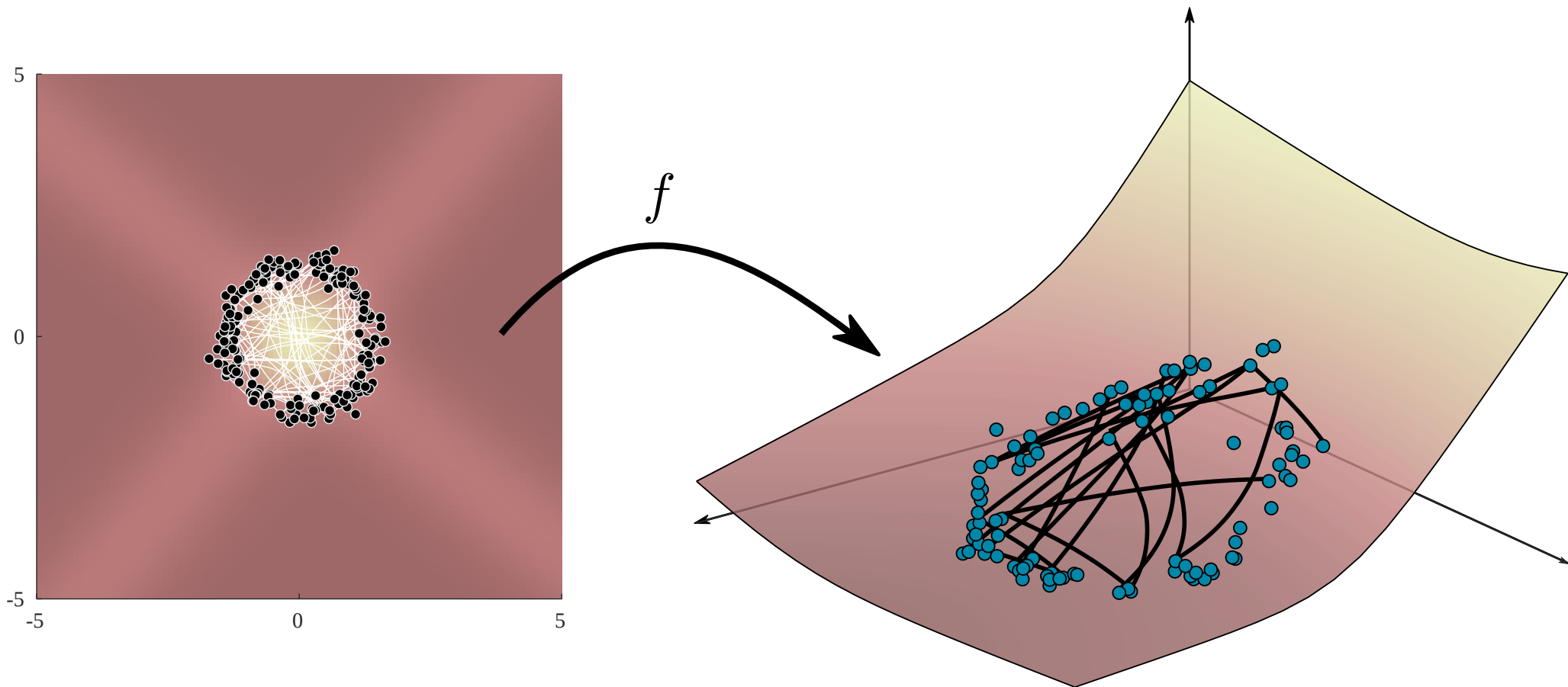
bit interesting!

$$\mathbf{G}_{\mathbf{x}} = \underbrace{\mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x})}_{\text{Mean term}} + \underbrace{\mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x})}_{\text{Uncertainty term}}.$$



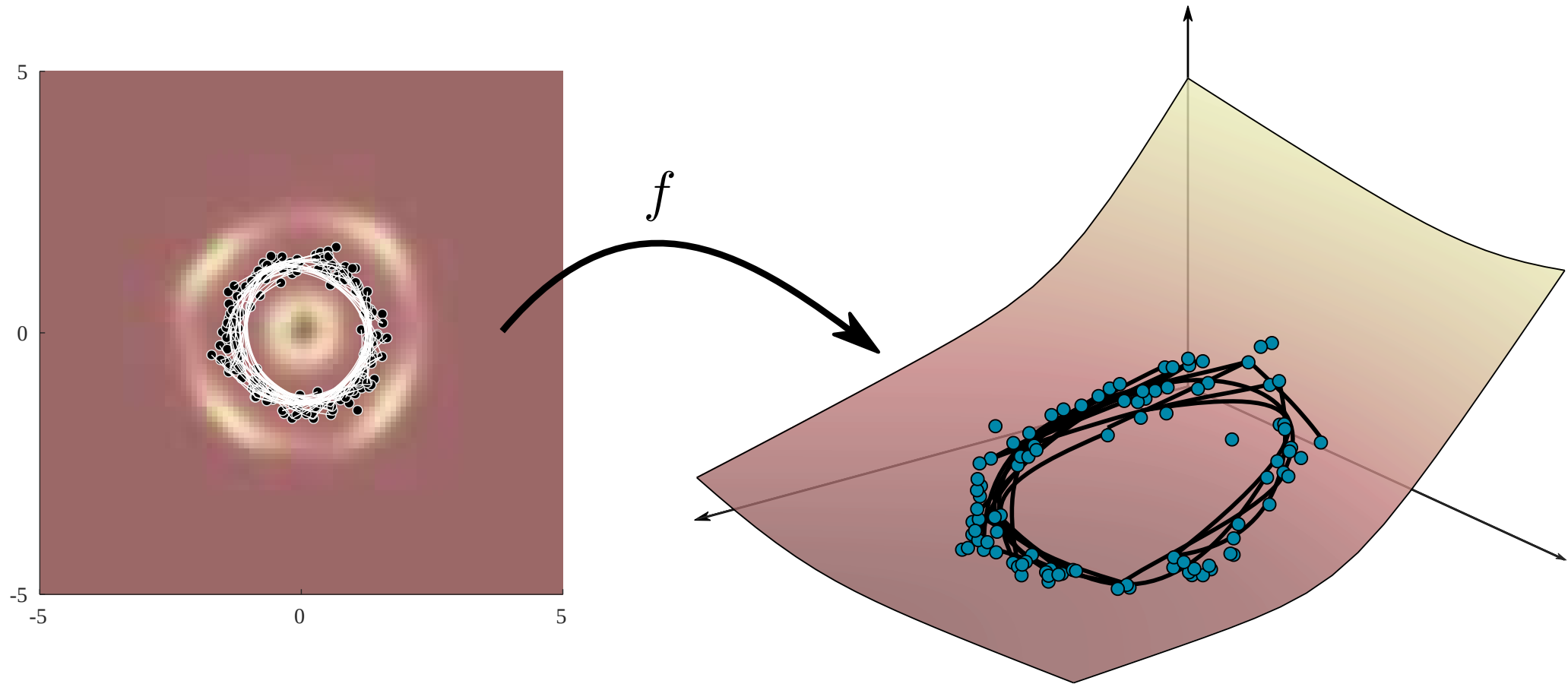
bit interesting.

$$\mathbf{G}_{\mathbf{x}} = \underbrace{\mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x})}_{\text{Mean term}} + \underbrace{\mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x})}_{\text{Uncertainty term}}.$$



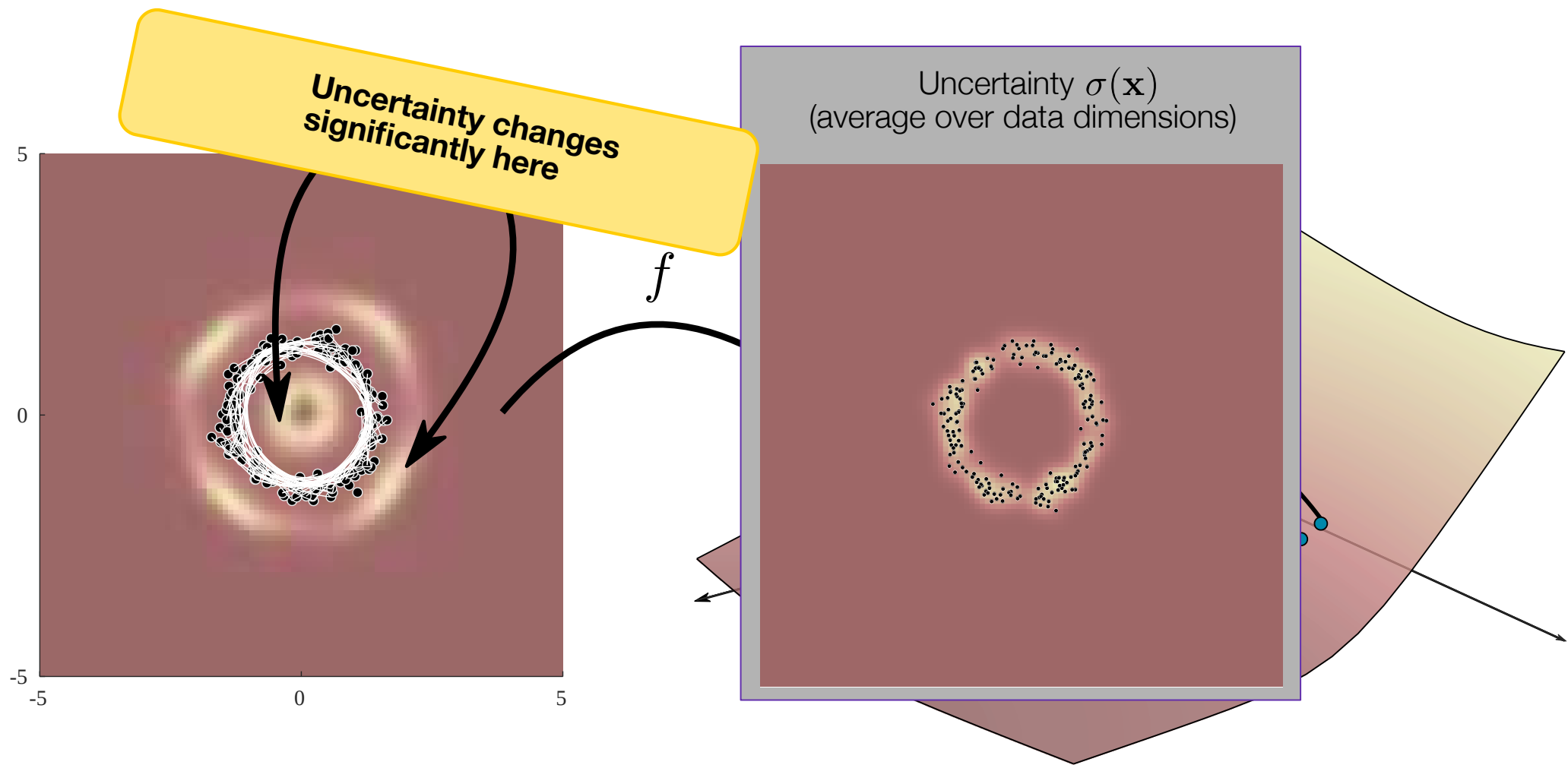
bit interesting!

$$\mathbf{G}_{\mathbf{x}} = \underbrace{\mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x})}_{\text{Mean term}} + \underbrace{\mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x})}_{\text{Uncertainty term}}.$$



...bit interesting!

$$\mathbf{G}_{\mathbf{x}} = \underbrace{\mathbf{J}_f(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x})}_{\text{Mean term}} + \underbrace{\mathbf{J}_\sigma(\mathbf{x})^\top \mathbf{J}_\sigma(\mathbf{x})}_{\text{Uncertainty term}}.$$



Intermediate summary

Riemannian geometry does not apply directly to stochastic models of the form

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \sigma(\mathbf{x}),$$

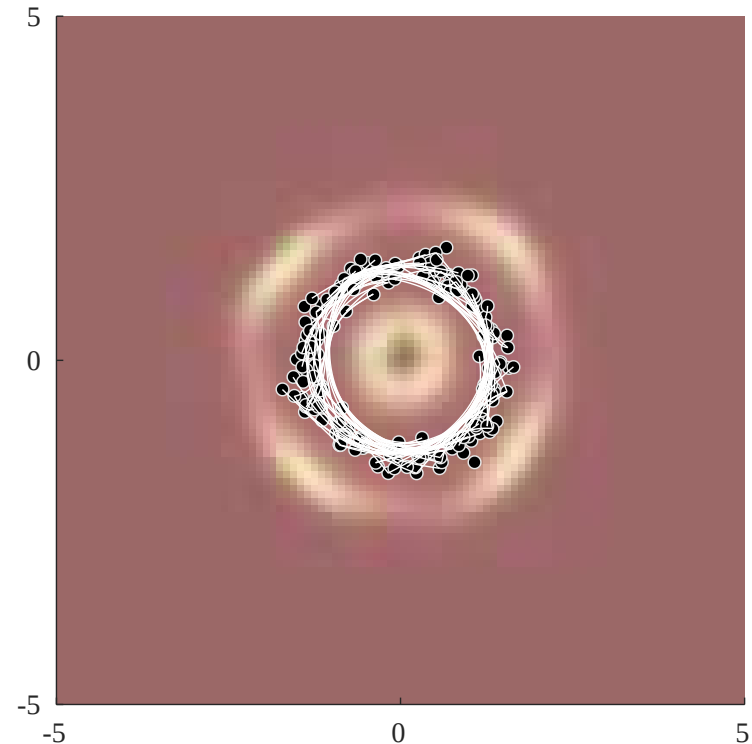
due to the stochasticity.

But if we re-interpret the model as a random projection of a deterministic manifold

$$= \begin{pmatrix} \mathbf{I} & \epsilon \end{pmatrix} \begin{pmatrix} f(\mathbf{x}) \\ \sigma(\mathbf{x}) \end{pmatrix} = \mathbf{P}h(\mathbf{x})$$

then we can study the geometry of the pre-projected manifold using the classic tools. Neat trick!

If uncertainty is well-calibrated (in neural networks, it usually is not), then it builds a 'wall' around the data, such that geodesics follow the trends.



Ensembles

We saw that we need to estimate uncertainty to capture topology

The simplest to capture model uncertainty is to use **ensembles**. In the case of a VAE, this amounts to having multiple decoders networks with different initialization.

We saw that we need to estimate uncertainty to capture topology

The simplest to capture model uncertainty is to use **ensembles**. In the case of a VAE, this amounts to having multiple decoders networks with different initialization.

We, thus, initialize M random neural network decoders

$$f_1, \dots, f_M$$

and during VAE training we just sample a decoder uniformly

$$f \sim \{f_1, \dots, f_M\}$$

in each iteration, and do a gradient update as usual.

We saw that we need to estimate uncertainty to capture topology

To compute the energy of a curve in the ensemble model, we also sample decoders

$$\mathcal{E}(c) \approx \sum_{i=0}^N \mathbb{E}_{l,k} \left\| f_l(c(t_i)) - f_k(c(t_{i+1})) \right\|^2 ,$$

where the expectation is uniform over $\{1, \dots, M\}$

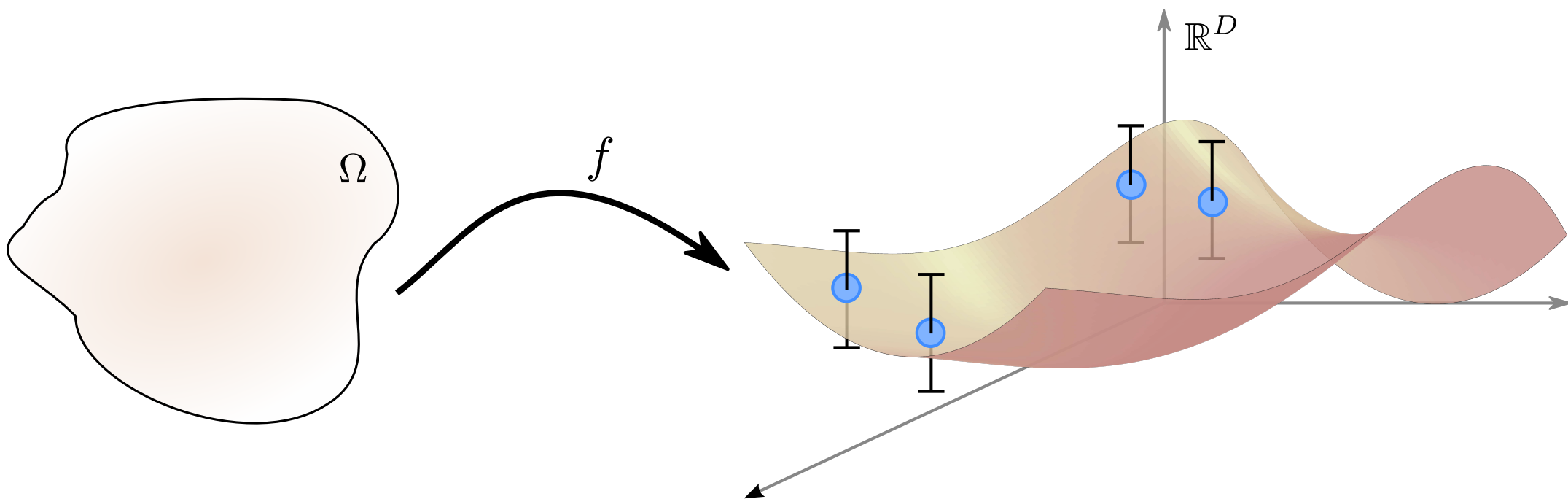
You'll be implementing this in the second mini-project (which starts today).

Information geometry

An alternative view

So far, we have considered the decoder to map from latent space to observation space, i.e.

$$\mathbf{y} = f(\mathbf{x}) \quad f : \mathbb{R}^d \rightarrow \mathbb{R}^D, \quad d \leq D$$

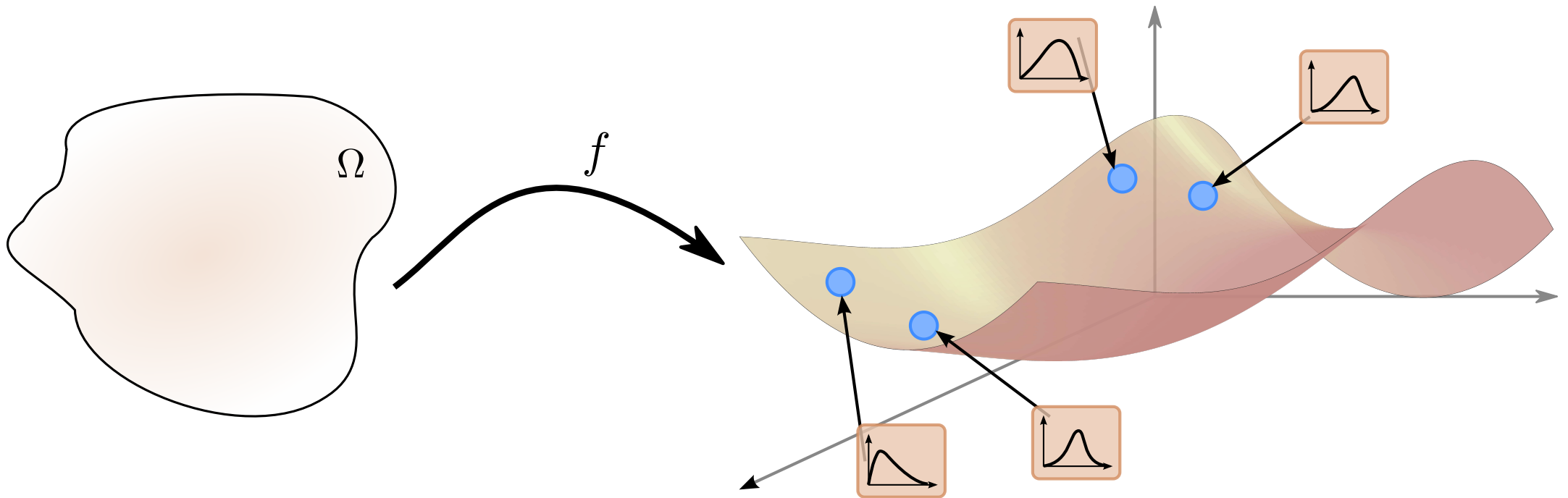


An alternative view

We will now switch to the view that the decoder maps to the space of conditional distributions, i.e.

$$f : \mathbf{x} \mapsto p(\mathbf{y}|\mathbf{x}).$$

We will let $\mathcal{P}$ denote the space of such distributions.



What is the space of distributions, again?

It sounds terribly abstract to speak of 'the space of distributions', but it's actually rather simple.

$$f : \mathbf{x} \mapsto p(\mathbf{y}|\mathbf{x}).$$

If the distribution $p(\mathbf{y}|\mathbf{x})$ is parameterized by θ then we call whichever space this variable lives 'the space of distributions'.

Distribution	θ	
Normal	(μ, σ)	$\mathbb{R} \times \mathbb{R}_+$
Multivariate normal	(μ, Σ)	$\mathbb{R}^D \times \mathbf{Sym}_+^{D \times D}$
Categorical	$(p_1, \dots, p_D)$	unit simplex
etc		

What is the space of distributions, again?

It sounds terribly abstract to speak of 'the space of distributions', but it's actually rather simple.

$$f : \mathbf{x} \mapsto p(\mathbf{y}|\mathbf{x}).$$

If the distribution $p(\mathbf{y}|\mathbf{x})$ is parameterized by θ then we call whichever space this variable lives 'the space of distributions'.

Distribution	θ	
Normal	(μ, σ)	$\mathbb{R} \times \mathbb{R}_+$
Multivariate normal	(μ, Σ)	$\mathbb{R}^D \times \text{Sym}_+^{D \times D}$
Categorical	$(p_1, \dots, p_D)$	unit simplex
etc		

But how do we measure distances in this space?

Consider, the example

$$p_1 = \mathcal{N}(\mu_1, \sigma_1^2)$$

$$p_2 = \mathcal{N}(\mu_2, \sigma_2^2)$$

If we choose the parametrization $\theta = (\mu, \sigma)$

which has Euclidean distance

$$\text{dist}(p_1, p_2) = \sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2}.$$

Consider, the example

$$p_1 = \mathcal{N}(\mu_1, \sigma_1^2)$$

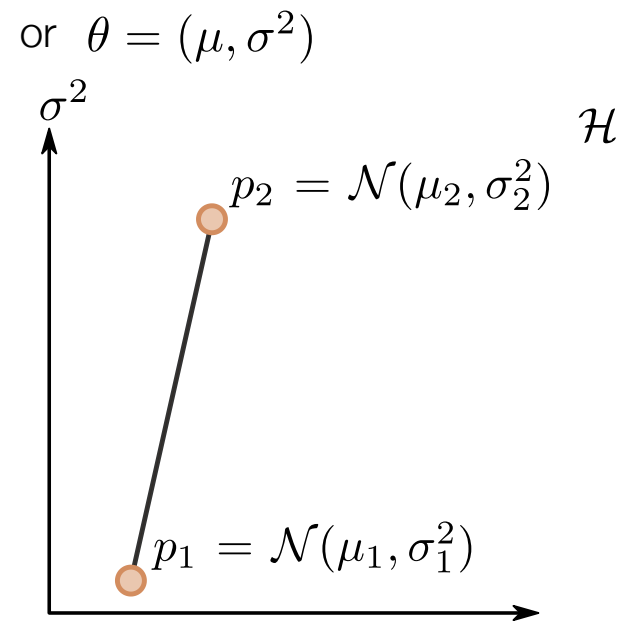
$$p_2 = \mathcal{N}(\mu_2, \sigma_2^2)$$

If we choose the parametrization $\theta = (\mu, \sigma)$

VS

which has Euclidean distance

$$\text{dist}(p_1, p_2) = \sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2}.$$



which has Euclidean distance

$$\text{dist}(p_1, p_2) = \sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1^2 - \sigma_2^2)^2}.$$

Consider, the example

$$p_1 = \mathcal{N}(\mu_1, \sigma_1^2)$$

$$p_2 = \mathcal{N}(\mu_2, \sigma_2^2)$$

If we choose

OH NO, NOT AGAIN....

One more time we face the issue that choice of parametrization matters, where it shouldn't.

To avoid making a choice, we really want a distance measure that is invariant to reparametrizations.

$\mathcal{H}$
 $\mathcal{N}(\mu_2, \sigma_2^2)$

(μ_1, σ_1^2)
→

which has Euclidean distance

which has Euclidean distance

$$\text{dist}(p_1, p_2) = \sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2}.$$

$$\text{dist}(p_1, p_2) = \sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1^2 - \sigma_2^2)^2}.$$

We have already met a divergence

We ran into the KL divergence between distributions when deriving the evidence lower bound,

$$\text{KL}(p\|q) = \int \log \left(\frac{p(\mathbf{y})}{q(\mathbf{y})} \right) p(\mathbf{y}) d\mathbf{y} = \mathbb{E}_{p(\mathbf{y})} \left[\log \left(\frac{p(\mathbf{y})}{q(\mathbf{y})} \right) \right]$$

This measures some form of similarity between distributions:

- It is invariant to choice of parametrization (why?)
- it's positive, convex, and only zero when comparing identical distributions
- but it is not symmetric, so it is not a distance.

Geodesics are energy minimizing curves

For our purposes, we just want to be able to compute the energy of a curve

$$\mathcal{E}(c) = \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{dist}^2 (f(c(t_i)), f(c(t_{i+1}))) .$$

Note how this only requires measuring distances locally...

Geodesics are energy minimizing curves

For our purposes, we just want to be able to compute the energy of a curve

$$\mathcal{E}(c) = \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{dist}^2 (f(c(t_i)), f(c(t_{i+1}))) .$$

Note how this only requires measuring distances locally...

We can look at the KL divergence locally by considering a small displacement in parameter space, i.e. let

$$p(\mathbf{y}) = p_{\theta}(\mathbf{y}) \qquad q(\mathbf{y}) = p_{\theta+\delta}(\mathbf{y})$$

Geodesics are energy minimizing curves

For our purposes, we just want to be able to compute the energy of a curve

$$\mathcal{E}(c) = \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{dist}^2 (f(c(t_i)), f(c(t_{i+1}))) .$$

Note how this only requires measuring distances locally...

We can look at the KL divergence locally by considering a small displacement in parameter space, i.e. let

$$p(\mathbf{y}) = p_{\theta}(\mathbf{y}) \qquad q(\mathbf{y}) = p_{\theta+\delta}(\mathbf{y})$$

Then we can make a second-order Taylor expansion of the KL

$$\text{KL}(p||q) = -\frac{1}{2} \delta^{\top} \underbrace{\int p(\mathbf{y}) \nabla_{\theta}^2 \log p(\mathbf{y}) d\mathbf{y}}_{\mathbf{F}} \delta + \mathcal{O}(\|\delta\|^3),$$

For our purposes, we just want to be able to compute the energy of a curve

$$\mathcal{E}(c) = \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{dist}^2 (f(c(t_i)), f(c(t_{i+1}))) .$$

Note how this only requires measuring distances locally...

We can look at the KL divergence locally by considering a small displacement in parameter space, i.e. let

$$p(\mathbf{y}) = p_{\theta}(\mathbf{y}) \qquad q(\mathbf{y}) = p_{\theta+\delta}(\mathbf{y})$$

Then we can make a second-order Taylor expansion of the KL

$$\text{KL}(p||q) = -\frac{1}{2} \delta^{\top} \underbrace{\int p(\mathbf{y}) \nabla_{\theta}^2 \log p(\mathbf{y}) d\mathbf{y}}_{\mathbf{F}} \delta + \mathcal{O}(\|\delta\|^3),$$

where ∇_{θ}^2 denotes the Hessian, such that locally the KL can be written

$$\text{KL}(p||q) = \delta^{\top} \mathbf{F} \delta.$$

Note how this only requires measuring distances locally...

We can look at the KL divergence locally by considering a small displacement in parameter space, i.e. let

$$p(\mathbf{y}) = p_{\theta}(\mathbf{y}) \qquad q(\mathbf{y}) = p_{\theta+\delta}(\mathbf{y})$$

Then we can make a second-order Taylor expansion of the KL

$$\text{KL}(p||q) = -\frac{1}{2}\delta^{\top} \underbrace{\int p(\mathbf{y}) \nabla_{\theta}^2 \log p(\mathbf{y}) d\mathbf{y}}_{\mathbf{F}} \delta + \mathcal{O}(\|\delta\|^3),$$

where ∇_{θ}^2 denotes the Hessian, such that locally the KL can be written

$$\text{KL}(p||q) = \delta^{\top} \mathbf{F} \delta.$$

We can provide a closer look at the matrix $\mathbf{F}$ (see exercise 12.3 in the book)

$$\begin{aligned} \mathbf{F} &= -\frac{1}{2} \int p(\mathbf{y}) \nabla^2 \log p(\mathbf{y}) d\mathbf{y} \\ &= \frac{1}{2} \int p(\mathbf{y}) \nabla_{\theta} \log p(\mathbf{y}) \nabla_{\theta} \log p(\mathbf{y})^{\top} d\mathbf{y} \end{aligned}$$

$$\text{KL}(p||q) = -\frac{1}{2}\delta^\top \underbrace{\int p(\mathbf{y}) \nabla_\theta^2 \log p(\mathbf{y}) d\mathbf{y}}_{\mathbf{F}} \delta + \mathcal{O}(\|\delta\|^3),$$

where ∇_θ^2 denotes the Hessian, such that locally the KL can be written

$$\text{KL}(p||q) = \delta^\top \mathbf{F} \delta.$$

We can provide a closer look at the matrix $\mathbf{F}$ (see exercise 12.3 in the book)

$$\begin{aligned} \mathbf{F} &= -\frac{1}{2} \int p(\mathbf{y}) \nabla^2 \log p(\mathbf{y}) d\mathbf{y} \\ &= \frac{1}{2} \int p(\mathbf{y}) \nabla_\theta \log p(\mathbf{y}) \nabla_\theta \log p(\mathbf{y})^\top d\mathbf{y} \end{aligned}$$

In summary,

- Locally, the KL divergence is a quadratic function.
- The Hessian is positive definite.
- The KL divergence, thus, can be seen as locally forming a Riemannian metric over the parameter space of the distribution.
- This Riemannian metric is known as the Fisher-Rao metric.

In summary,

- Locally, the KL divergence is a quadratic function.
- The Hessian is positive definite.
- The KL divergence, thus, can be seen as locally forming a Riemannian metric over the parameter space of the distribution.
- This Riemannian metric is known as the Fisher-Rao metric.

We can, thus, use the KL divergence locally to define a curve energy

$$\begin{aligned}\mathcal{E}(c) &= \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{KL}(f(c(t_i)) \parallel f(c(t_{i+1}))) \\ &= \int_0^1 \left\| \frac{d}{dt} f(c_t) \right\|_{\mathbf{F}_{c_t}}^2 dt \\ &= \int_0^1 \dot{c}_t^\top \mathbf{J}_{c_t}^\top \mathbf{F}_{c_t} \mathbf{J}_{c_t} \dot{c}_t dt.\end{aligned}$$

such that our latent space metric is now $\mathbf{G}_{c_t} = \mathbf{J}_{c_t}^\top \mathbf{F}_{c_t} \mathbf{J}_{c_t}$.

We can, thus, use the KL divergence locally to define a curve energy

$$\begin{aligned}\mathcal{E}(c) &= \lim_{N \rightarrow \infty} \sum_{i=0}^N \text{KL}(f(c(t_i)) \parallel f(c(t_{i+1}))) \\ &= \int_0^1 \left\| \frac{d}{dt} f(c_t) \right\|_{\mathbf{F}_{c_t}}^2 dt \\ &= \int_0^1 \dot{c}_t^\top \mathbf{J}_{c_t}^\top \mathbf{F}_{c_t} \mathbf{J}_{c_t} \dot{c}_t dt.\end{aligned}$$

such that our latent space metric is now $\mathbf{G}_{c_t} = \mathbf{J}_{c_t}^\top \mathbf{F}_{c_t} \mathbf{J}_{c_t}$.

Computationally, we can approximate the energy as

$$\mathcal{E}(c) \approx \sum_{i=0}^N \text{KL}(f(c(t_i)) \parallel f(c(t_{i+1}))),$$

and compute geodesics by minimizing this (just like last week).

Computationally, we can approximate the energy as

$$\mathcal{E}(c) \approx \sum_{i=0}^N \text{KL}(f(c(t_i)) \parallel f(c(t_{i+1}))),$$

and compute geodesics by minimizing this (just like last week).

And that's what I mean by this figure

Parameter manifolds

All parameters are latent

So far, we have been looking at latent variable models (like the VAE) and asked:
what is the geometry of the latent space?

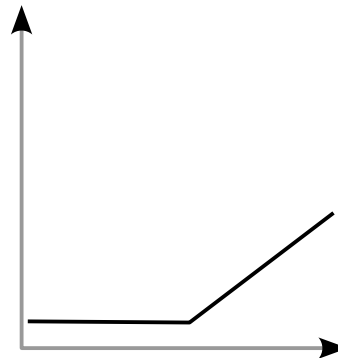
But, really, there is nothing special about *latent variables*, so we can ask the same question for any model parameter.

We will now do that for the parameters of a general neural network!

A toy-example

Consider the (absurdly) simple neural network

$$f(x) = w_1 \text{ReLU}(w_2 x); \quad f : \mathbb{R} \rightarrow \mathbb{R}.$$



A toy-example

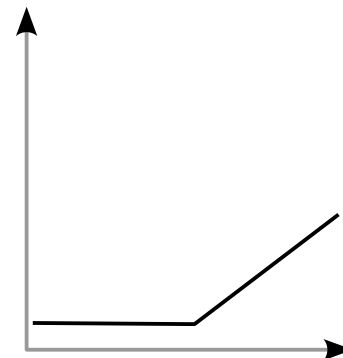
Consider the (absurdly) simple neural network

$$f(x) = w_1 \text{ReLU}(w_2 x); \quad f : \mathbb{R} \rightarrow \mathbb{R}.$$

Note that we can write this as

$$f(x) = w_1/\alpha \text{ReLU}(\alpha w_2 x)$$

This implies that the weights (w_1, w_2) and $(w_1/\alpha, \alpha w_2)$ correspond to the same function.



A toy-example

Consider the (absurdly) simple neural network

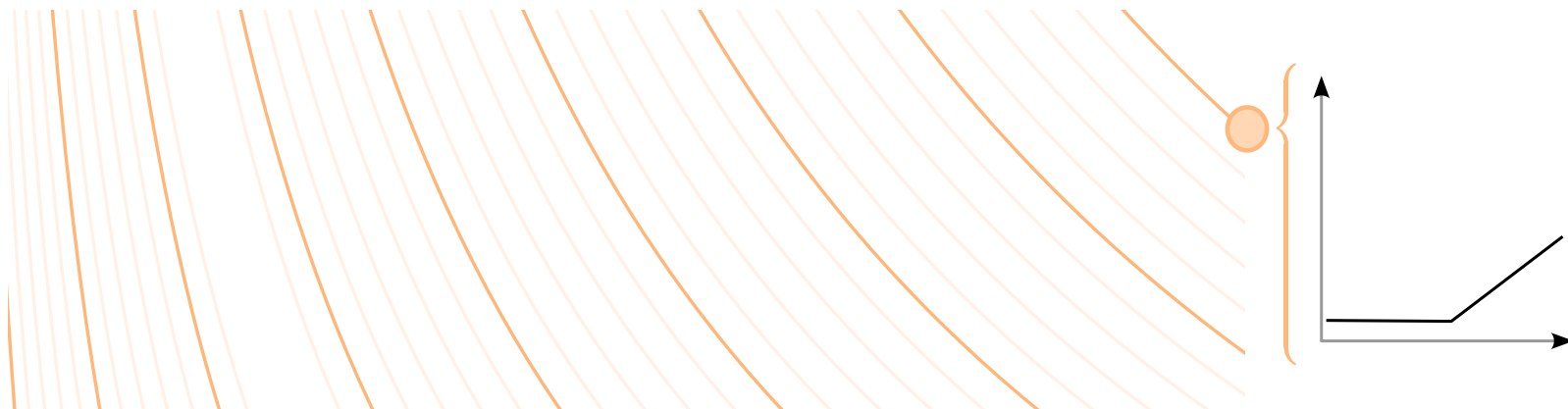
$$f(x) = w_1 \text{ReLU}(w_2 x); \quad f: \mathbb{R} \rightarrow \mathbb{R}.$$

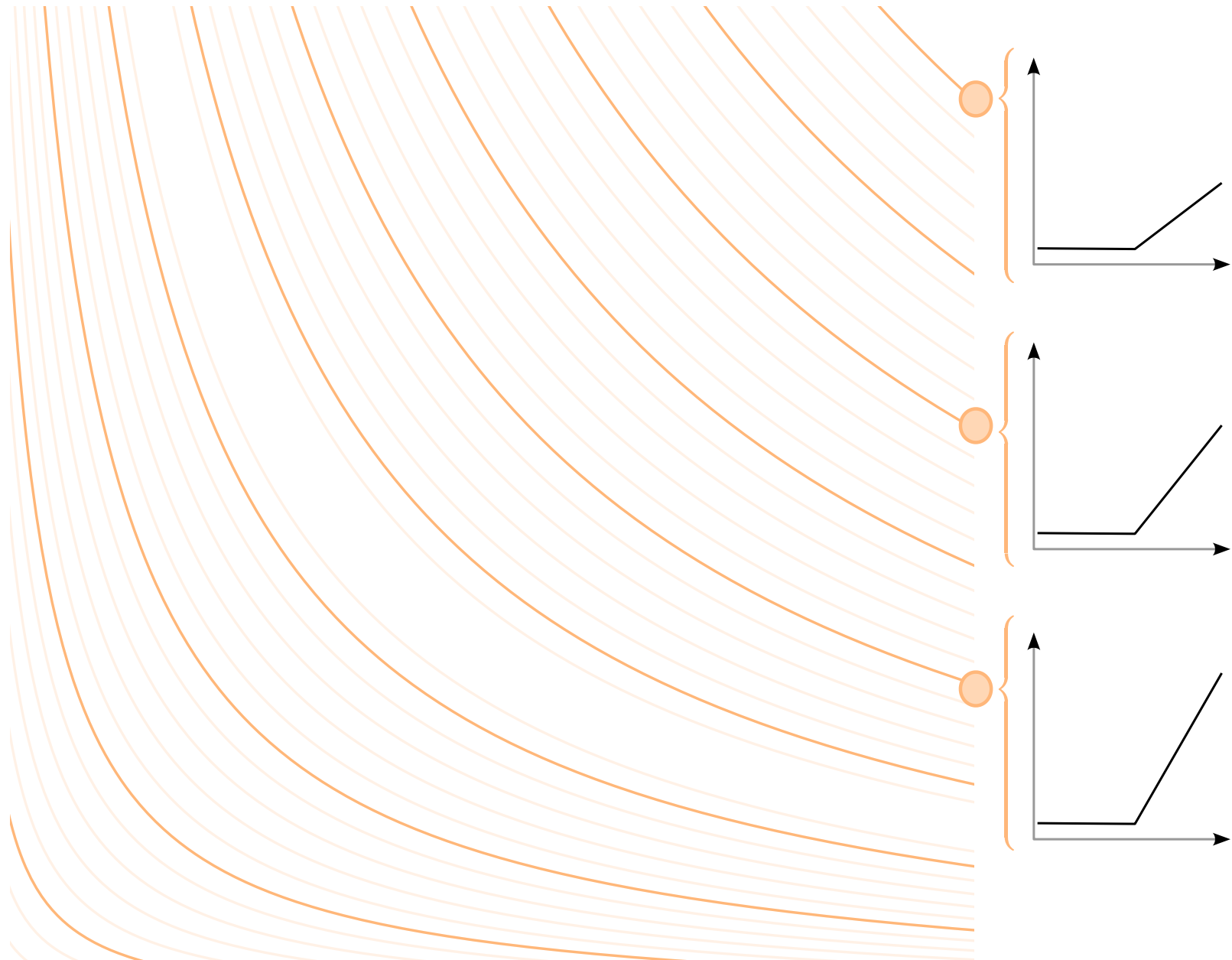
Note that we can write this as

$$f(x) = w_1/\alpha \text{ReLU}(\alpha w_2 x)$$

This implies that the weights (w_1, w_2) and $(w_1/\alpha, \alpha w_2)$ correspond to the same function.

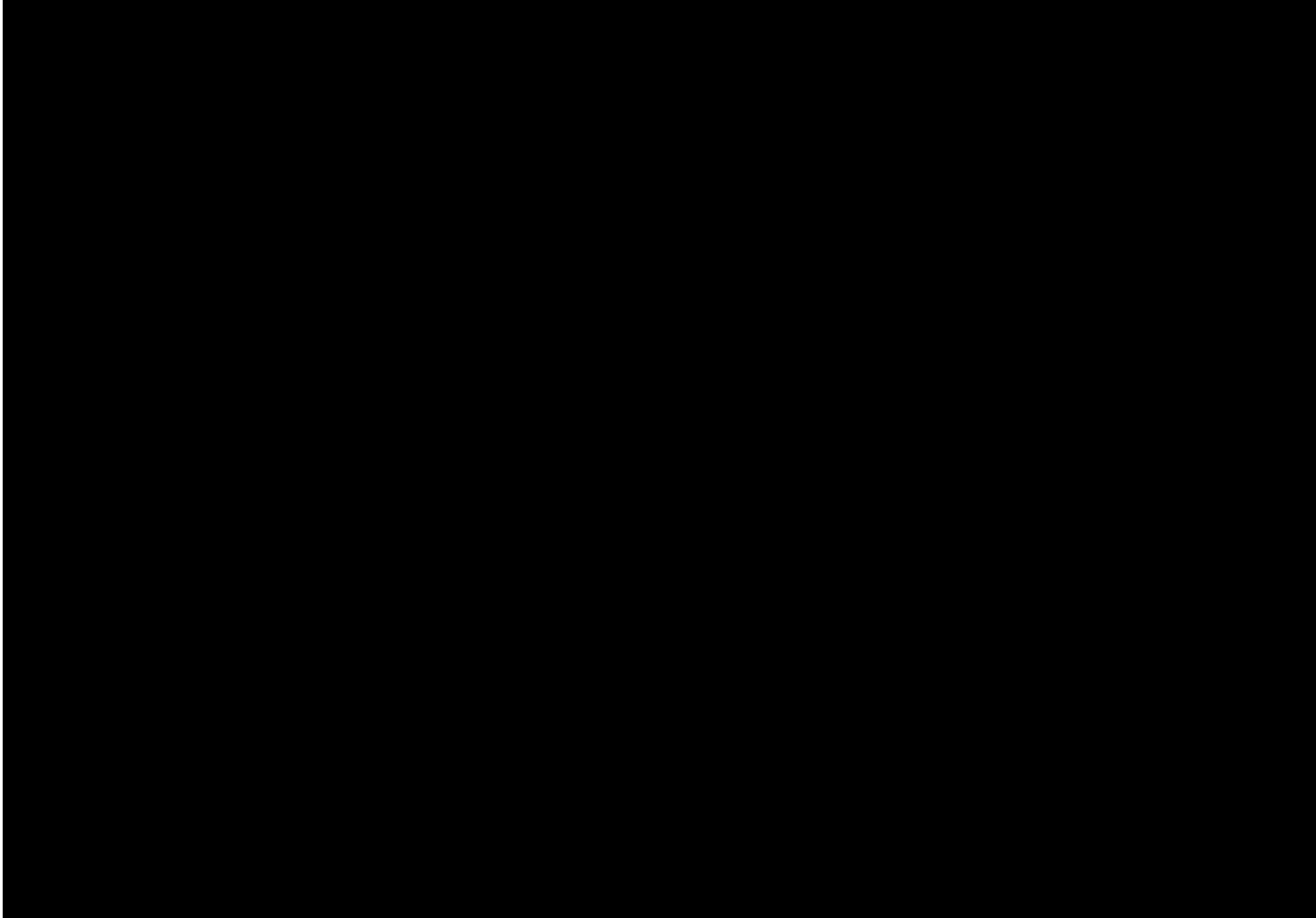
Let's look at this in weight space:





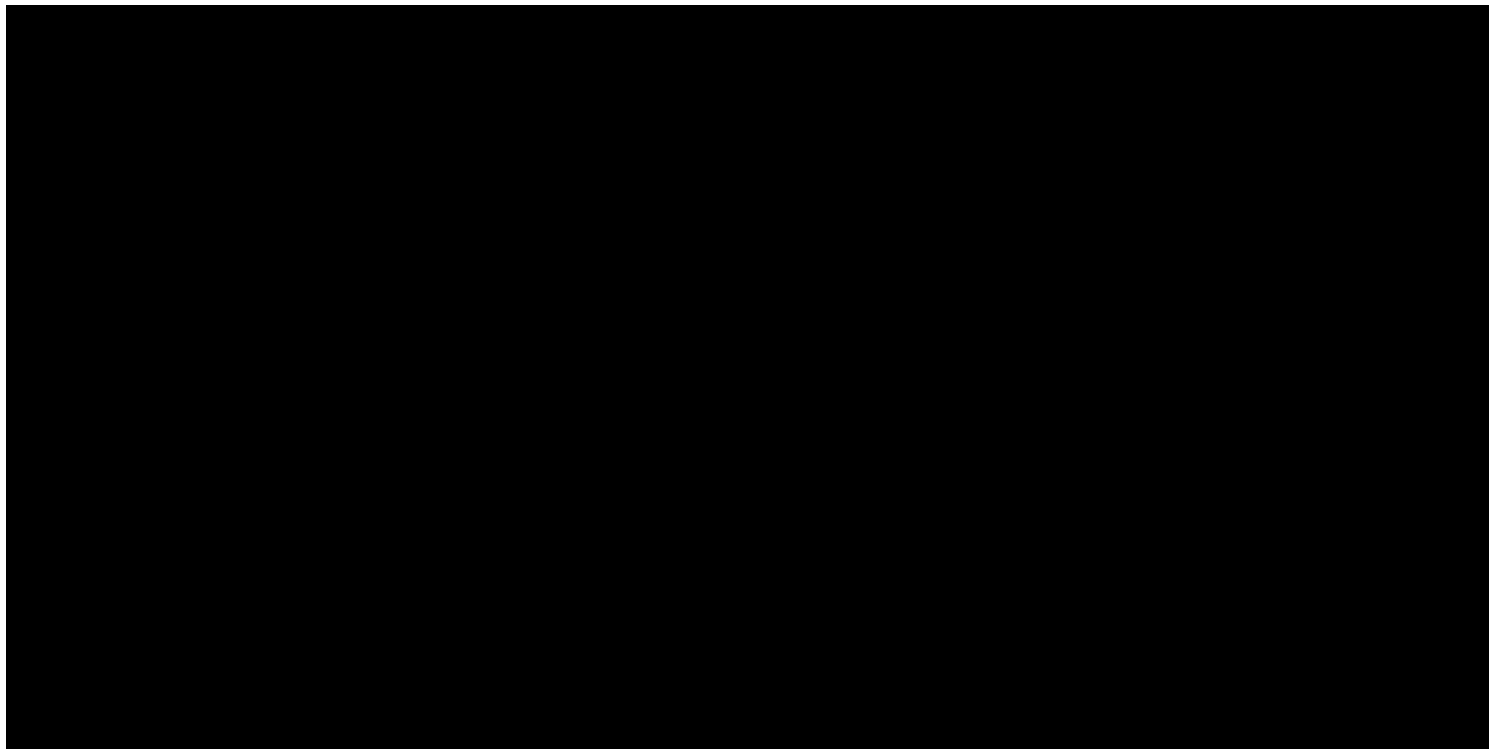
Adding more layers allow for fancy animations:

$$f_l(\mathbf{x}) = \text{ReLU}(w_l \mathbf{x}),$$

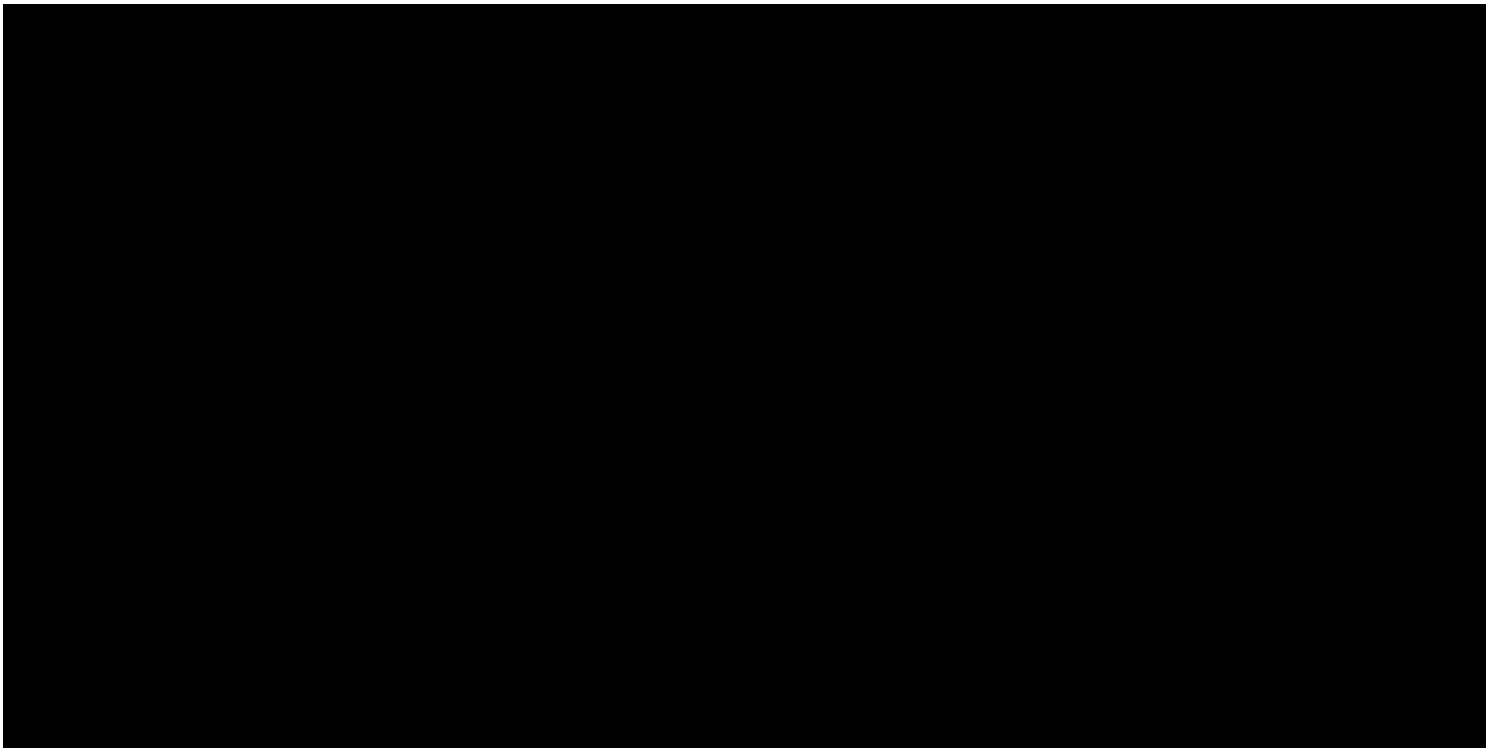
$$f(\mathbf{x}) = (f_L \circ \dots \circ f_1)(\mathbf{x}) = f_L(\dots f_2(f_1(\mathbf{x}))) \quad f : \mathbb{R} \rightarrow \mathbb{R}.$$


$$f(\mathbf{x}) = (f_L \circ \dots \circ f_1)(\mathbf{x}) = f_L(\dots f_2(f_1(\mathbf{x})))$$

$$f : \mathbb{R} \rightarrow \mathbb{R}.$$



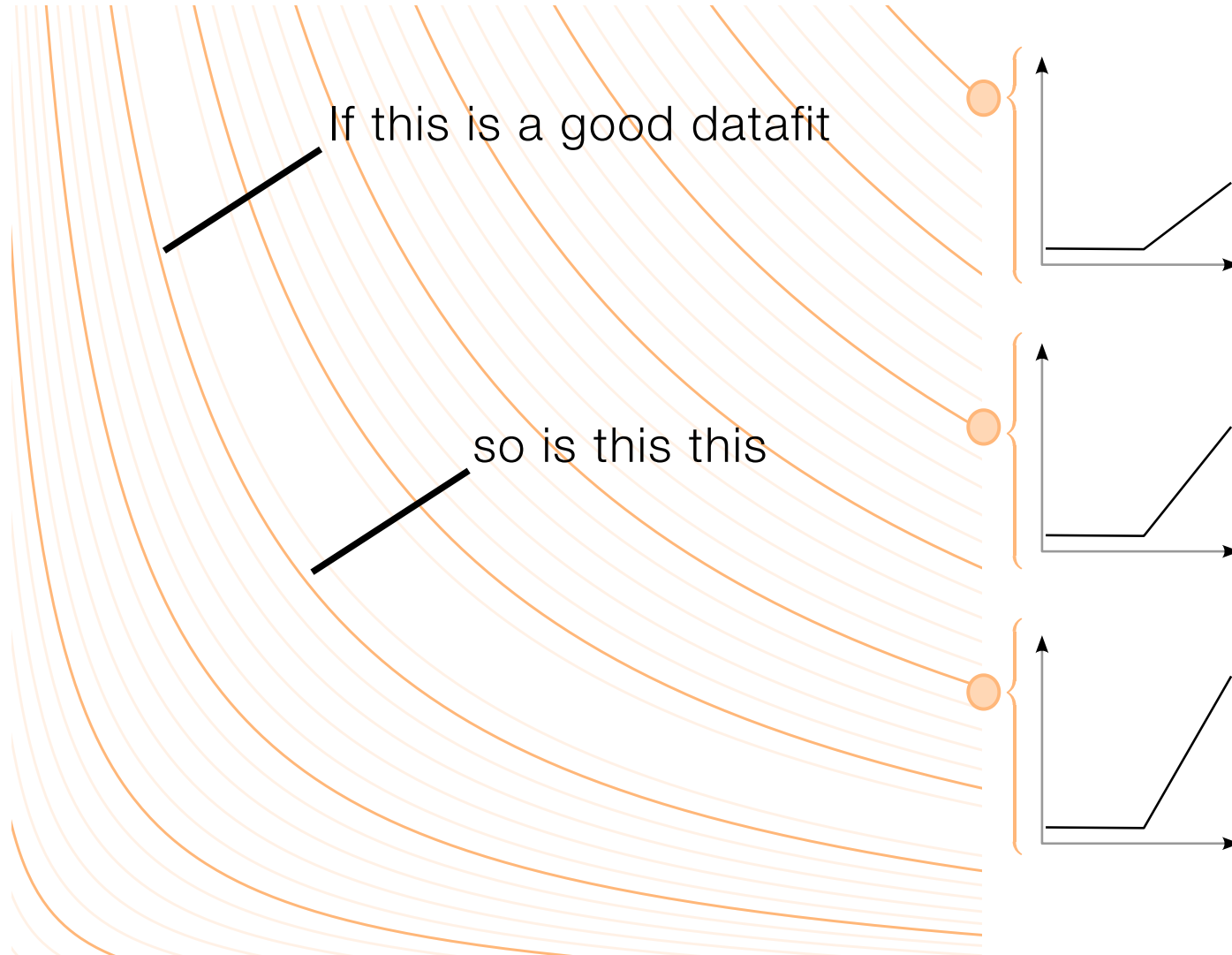
Changing the architecture will change how these surfaces look,



Changing the architecture will change how these surfaces look,
but they will practically always be there in some form.

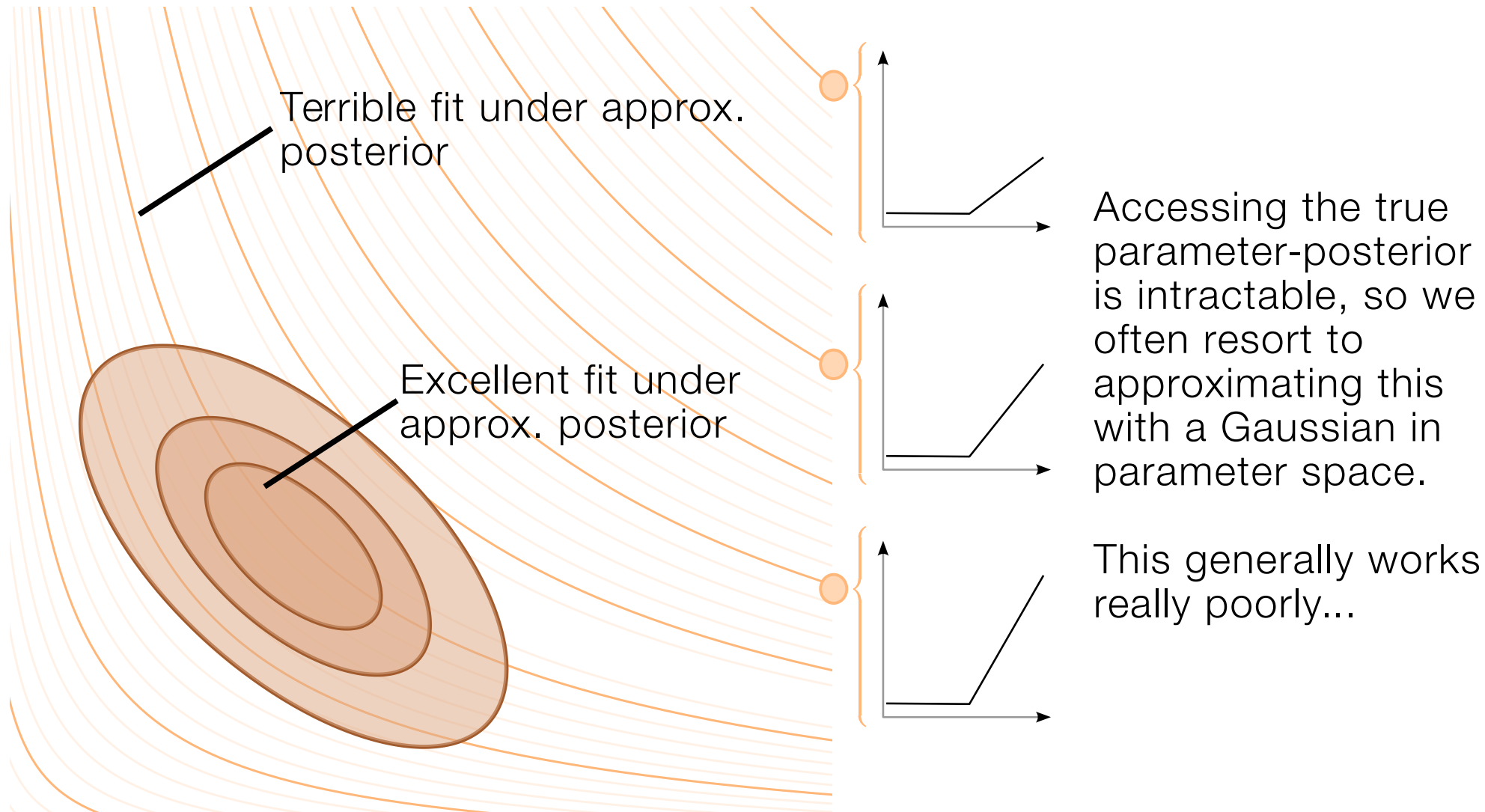
But does it matter?

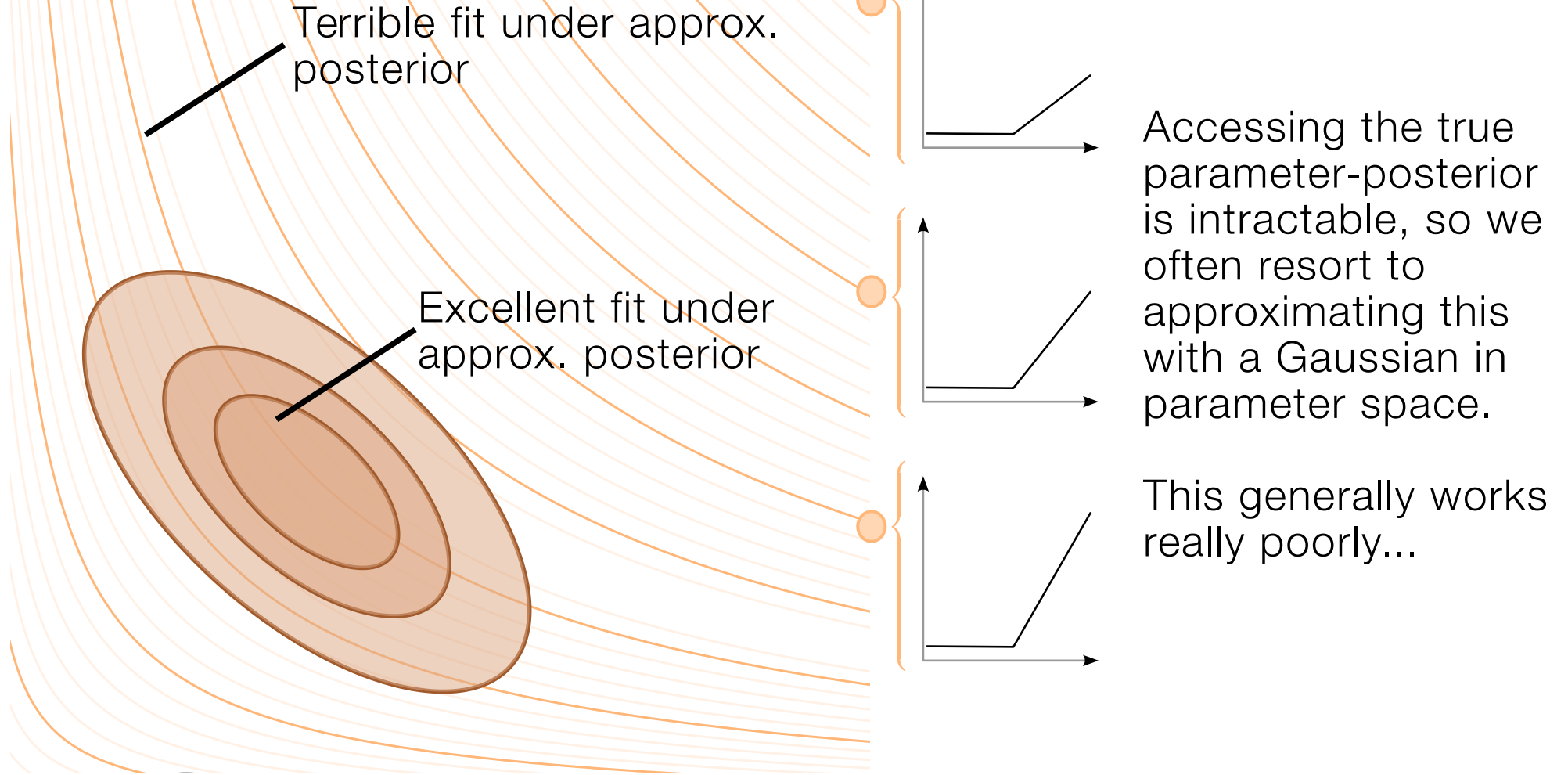
Optimization just got easier



Even if the parameter space is high-dimensional (suggesting that optimization is hard), we have a gigantic target, which helps explaining why deep learning isn't that hard after all.

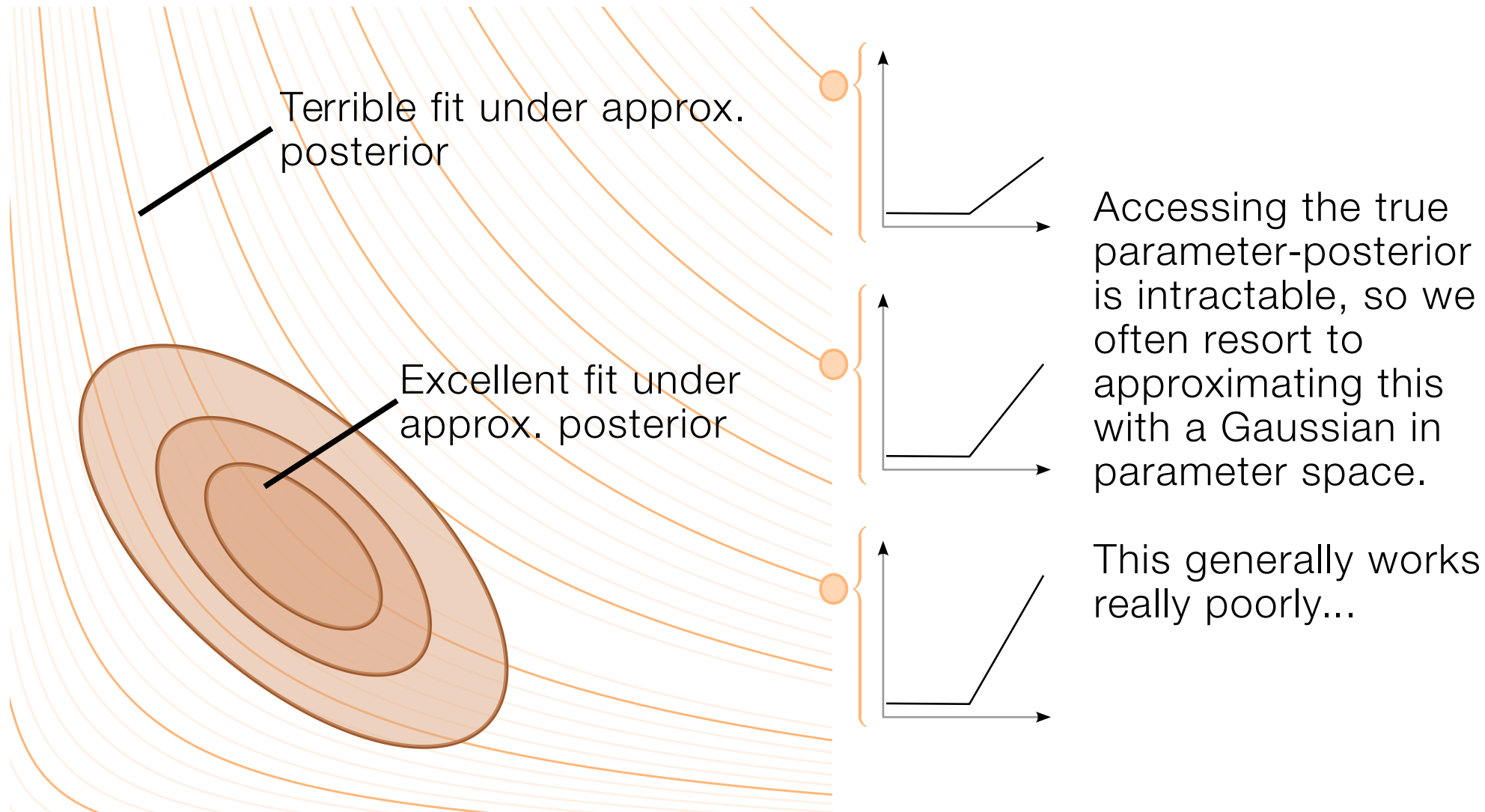
Being Bayesian just got a lot harder



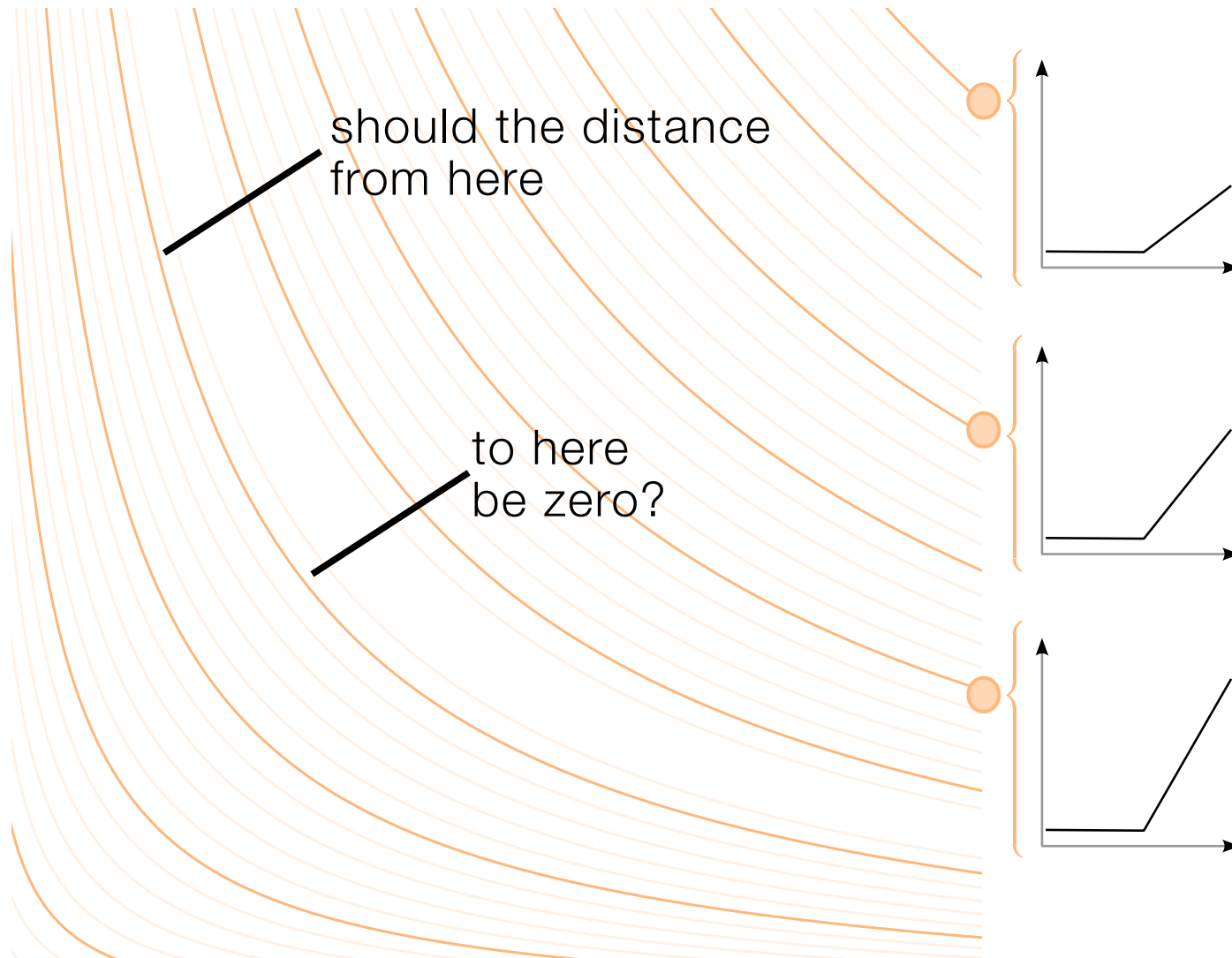


It would be really nice if the parameter space wasn't so damn weird....

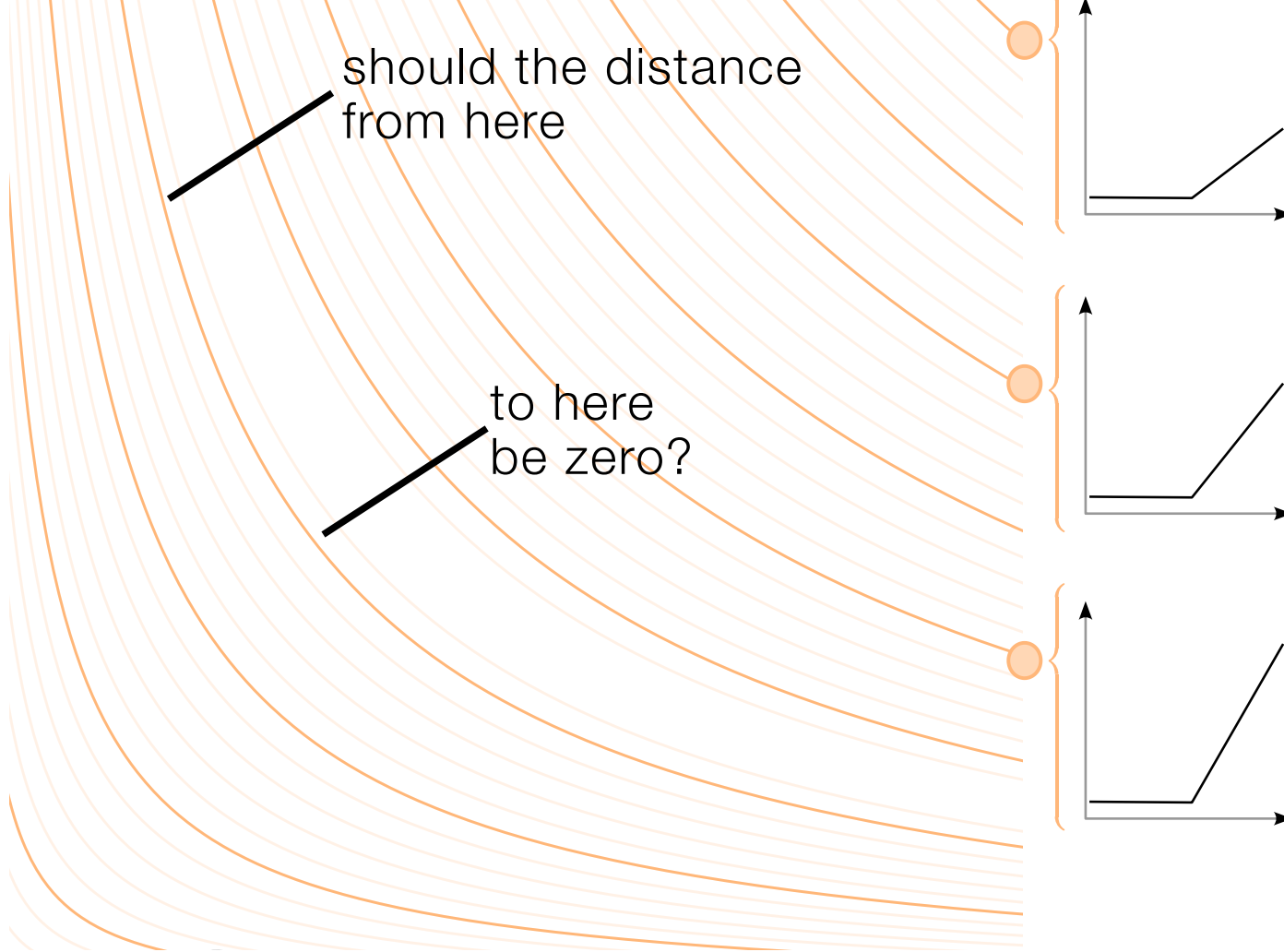
Being Bayesian just got a lot harder



Geometry to the rescue?



The most natural way to think neural networks is to think about the functions they represent.

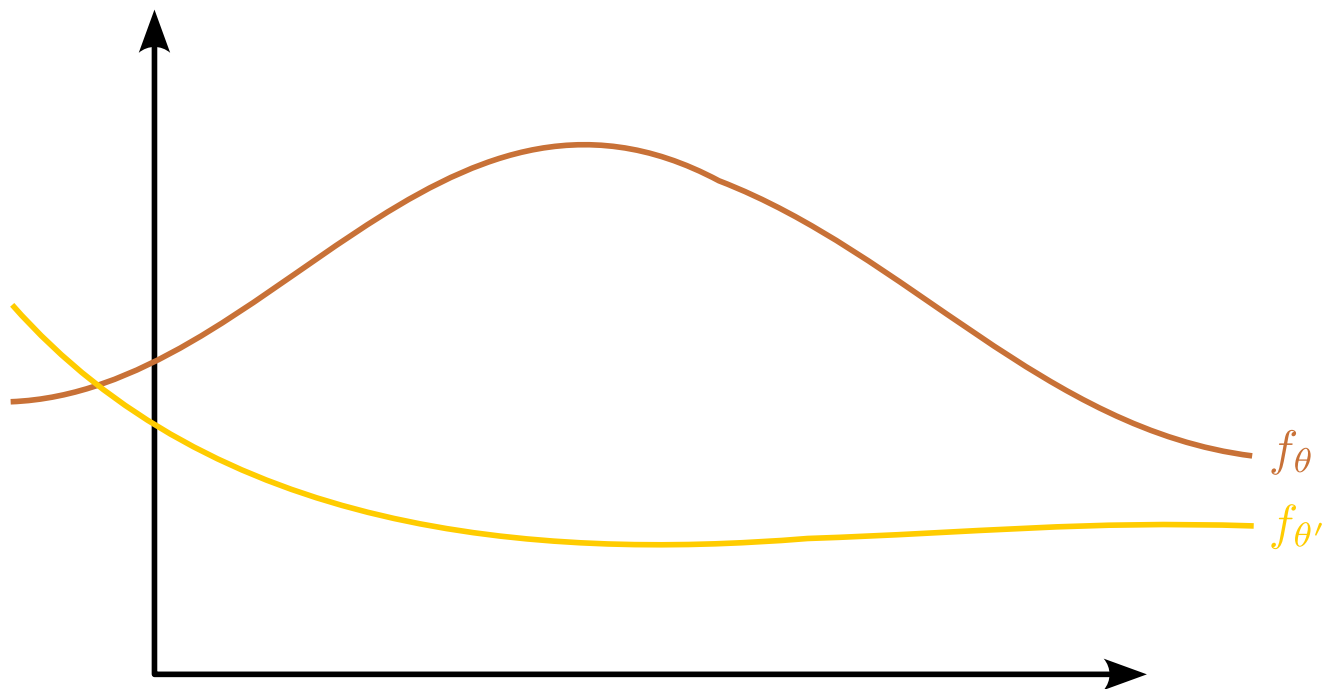


The most natural way to think neural networks is to think about the functions they represent.

Strategy: we will define an inner product in *function space* and bring this into *parameter space* using pullback metrics.

Inner product on functions

Consider two functions (neural networks)

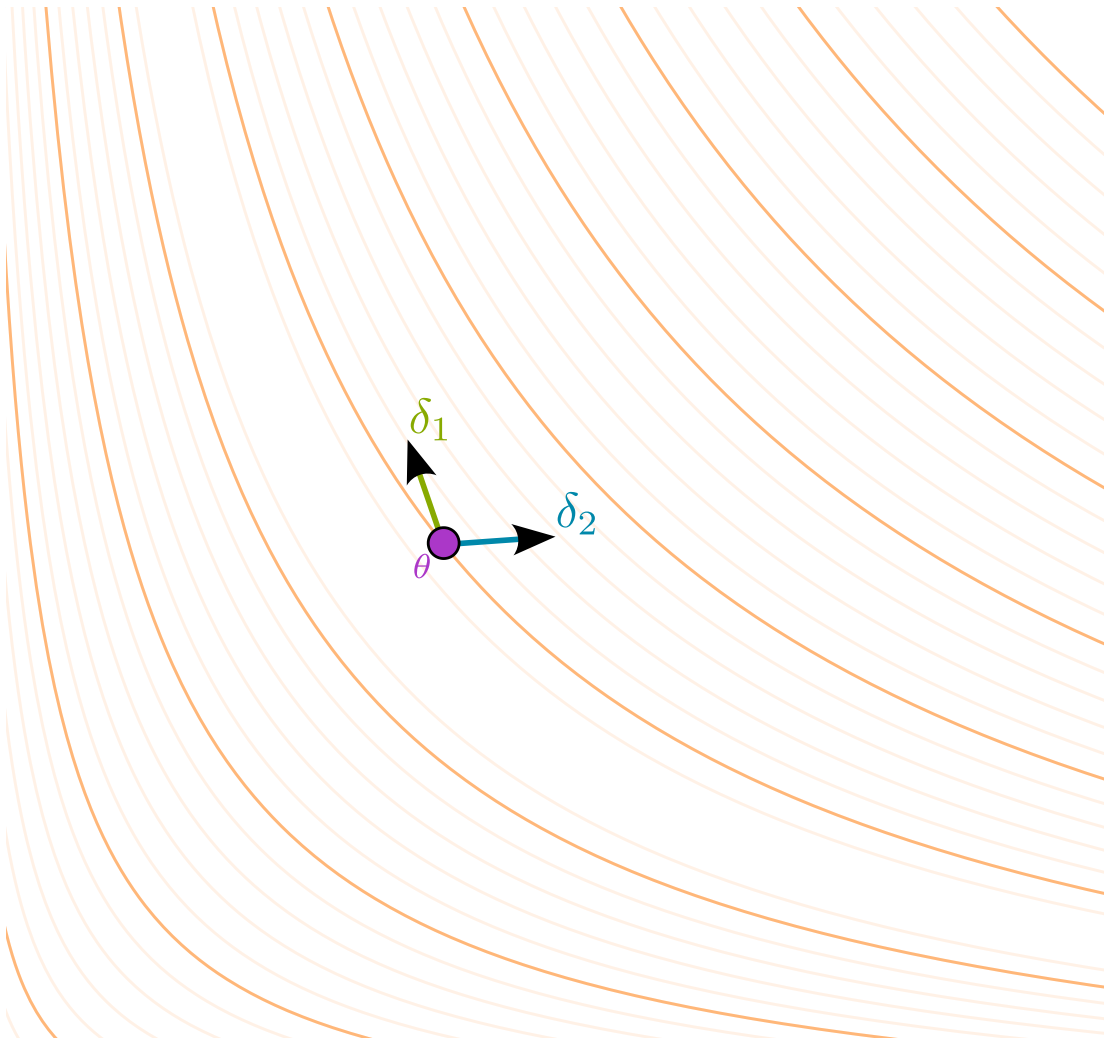


then we can define an inner product between them as the expected inner product between function outputs

$$\langle f_\theta, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_\theta(\mathbf{x})^\top f_{\theta'}(\mathbf{x})] .$$

then we can define an inner product between them as the expected inner product between function outputs

$$\langle f_{\theta}, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_{\theta}(\mathbf{x})^{\top} f_{\theta'}(\mathbf{x})] .$$

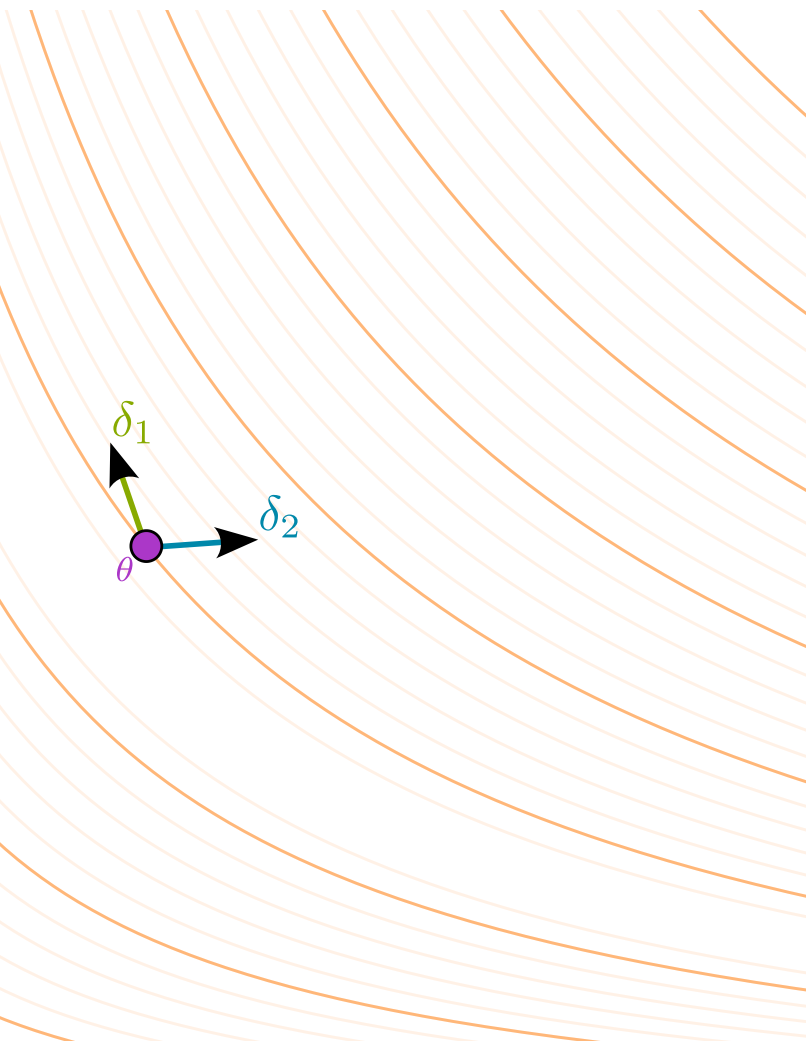


Locally, we can bring this inner product into the parameter space as

$$\langle \delta_1, \delta_2 \rangle_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta+\delta_1}(\mathbf{x}) - f_{\theta}(\mathbf{x}))^{\top} (f_{\theta+\delta_2}(\mathbf{x}) - f_{\theta}(\mathbf{x}))]$$

can define an inner product between them as the
d inner product between function outputs

$$\langle f_{\theta}, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_{\theta}(\mathbf{x})^{\top} f_{\theta'}(\mathbf{x})].$$

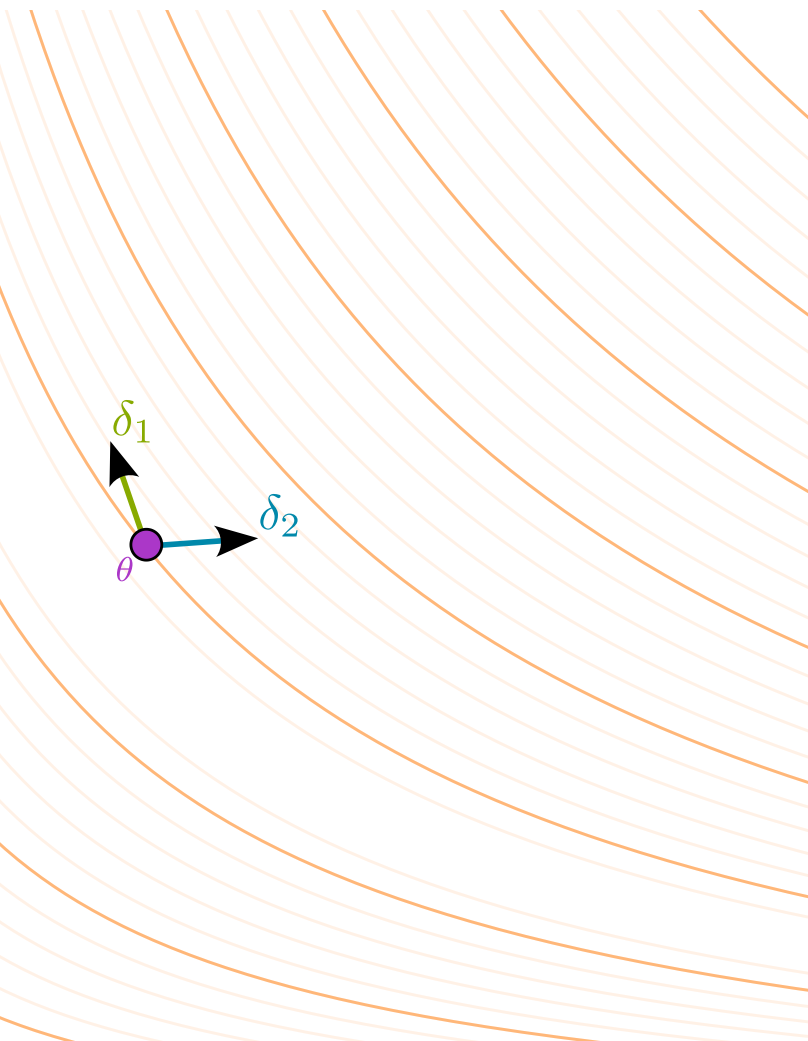


Locally, we can bring
this inner product into
the parameter space as

$$\langle \delta_1, \delta_2 \rangle_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta + \delta_1}(\mathbf{x}) - f_{\theta}(\mathbf{x}))^{\top} (f_{\theta + \delta_2}(\mathbf{x}) - f_{\theta}(\mathbf{x}))]$$

can define an inner product between them as the
d inner product between function outputs

$$\langle f_\theta, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_\theta(\mathbf{x})^\top f_{\theta'}(\mathbf{x})].$$



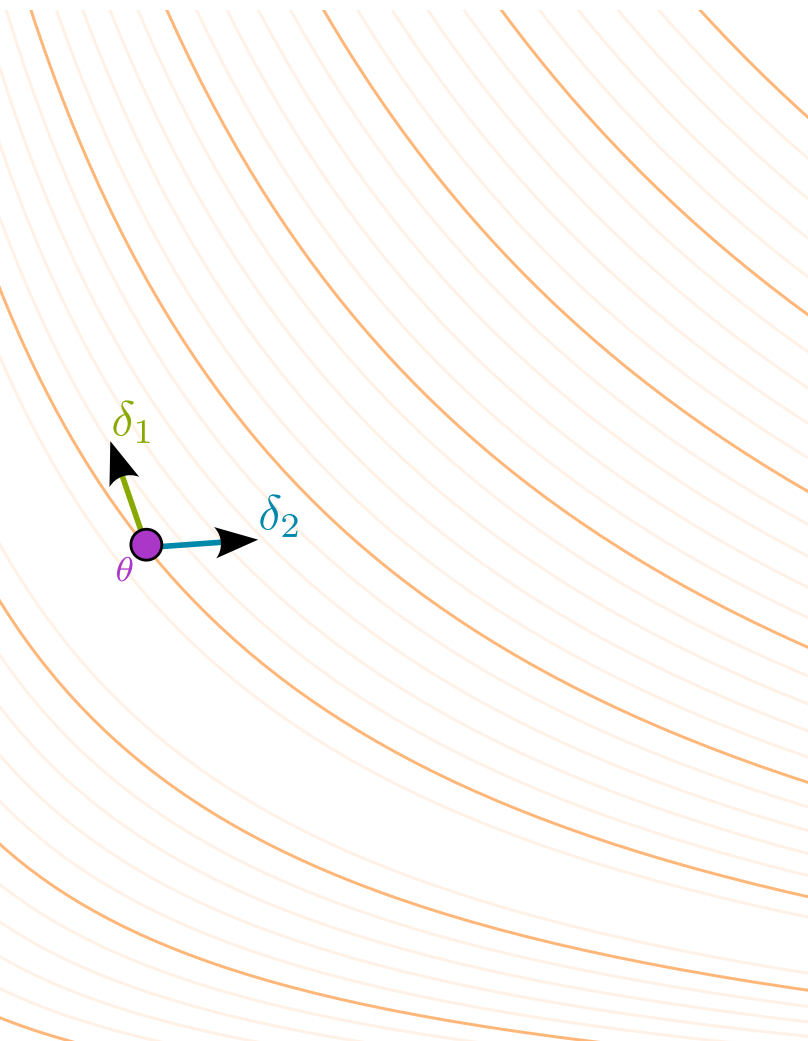
Locally, we can bring
this inner product into
the parameter space as

$$\begin{aligned} \langle \delta_1, \delta_2 \rangle_\theta &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta+\delta_1}(\mathbf{x}) - f_\theta(\mathbf{x}))^\top (f_{\theta+\delta_2}(\mathbf{x}) - f_\theta(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(\mathbf{J}_\theta(\mathbf{x})\delta_1)^\top (\mathbf{J}_\theta(\mathbf{x})\delta_2)] \end{aligned}$$

$$\mathbf{J}_\theta(\mathbf{x}) = \partial f_\theta(\mathbf{x}) / \partial \theta \in \mathbb{R}^{D \times P}$$

can define an inner product between them as the
d inner product between function outputs

$$\langle f_\theta, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_\theta(\mathbf{x})^\top f_{\theta'}(\mathbf{x})].$$



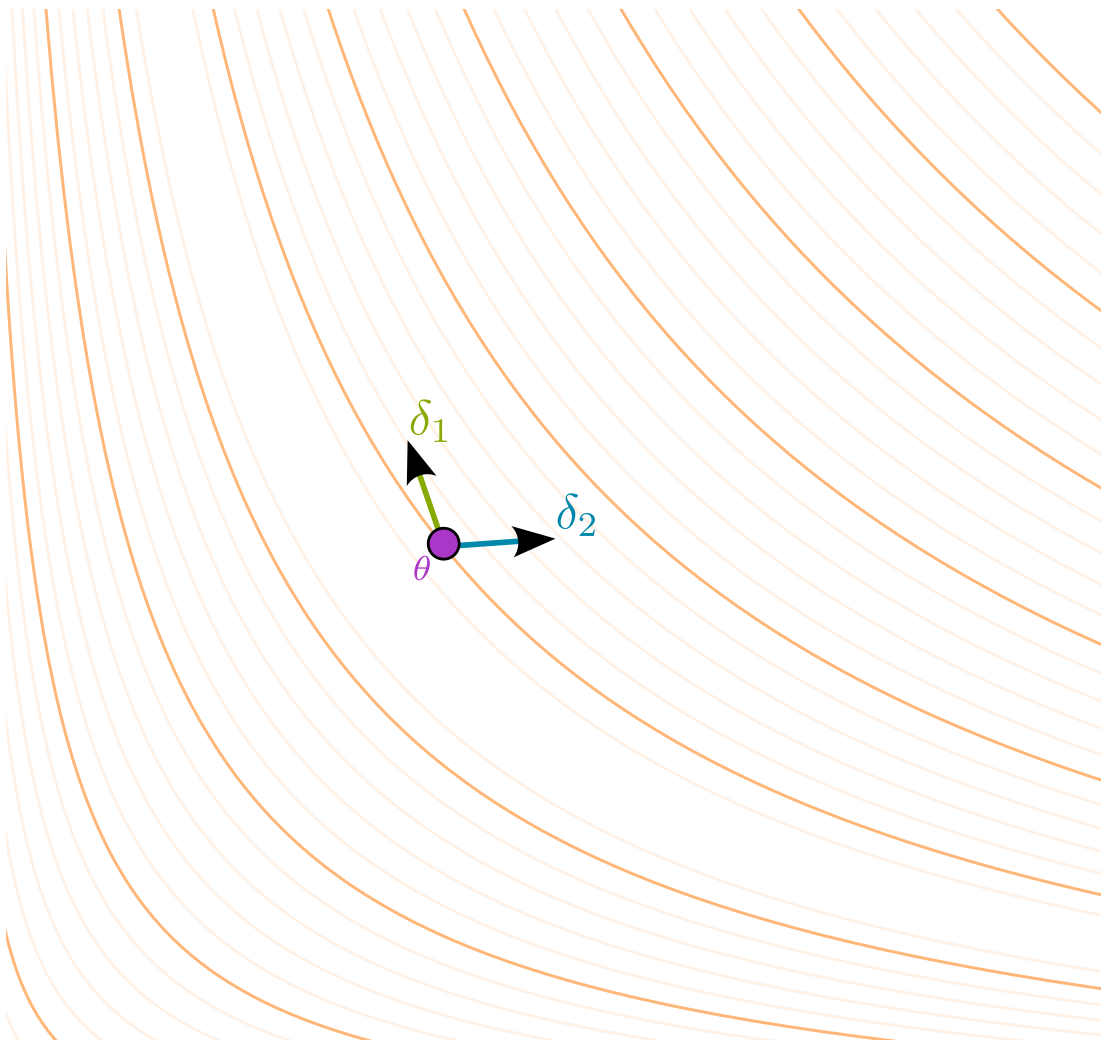
Locally, we can bring
this inner product into
the parameter space as

$$\begin{aligned} \langle \delta_1, \delta_2 \rangle_\theta &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta+\delta_1}(\mathbf{x}) - f_\theta(\mathbf{x}))^\top (f_{\theta+\delta_2}(\mathbf{x}) - f_\theta(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(\mathbf{J}_\theta(\mathbf{x}) \delta_1)^\top (\mathbf{J}_\theta(\mathbf{x}) \delta_2)] \\ &= \delta_1^\top \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_\theta(\mathbf{x})^\top \mathbf{J}_\theta(\mathbf{x})] \delta_2. \end{aligned}$$

$$\mathbf{J}_\theta(\mathbf{x}) = \partial f_\theta(\mathbf{x}) / \partial \theta \in \mathbb{R}^{D \times P}$$

then we can define an inner product between them as the expected inner product between function outputs

$$\langle f_{\theta}, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_{\theta}(\mathbf{x})^{\top} f_{\theta'}(\mathbf{x})] .$$



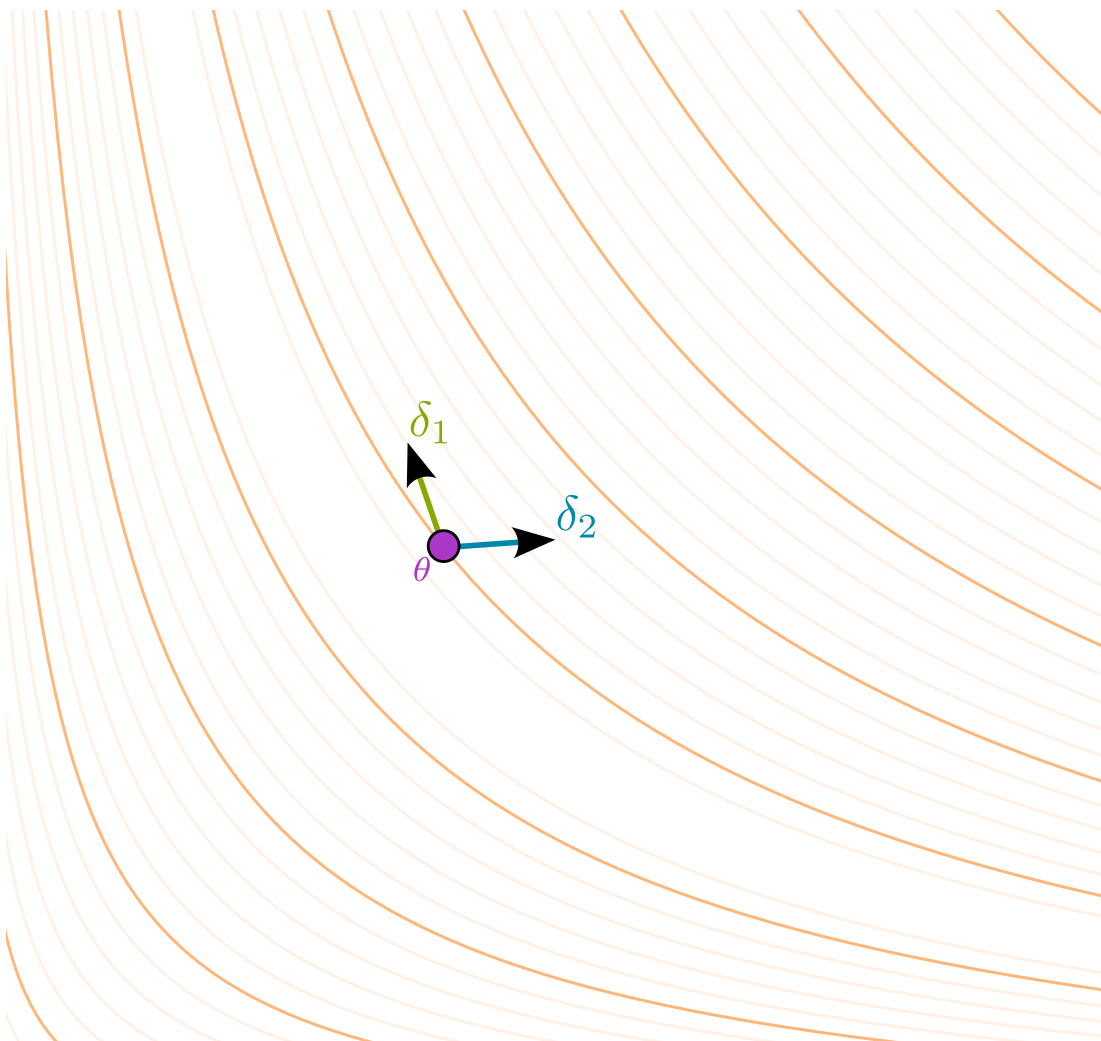
Locally, we can bring this inner product into the parameter space as

$$\begin{aligned} \langle \delta_1, \delta_2 \rangle_{\theta} &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta+\delta_1}(\mathbf{x}) - f_{\theta}(\mathbf{x}))^{\top} (f_{\theta+\delta_2}(\mathbf{x}) - f_{\theta}(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(\mathbf{J}_{\theta}(\mathbf{x}) \delta_1)^{\top} (\mathbf{J}_{\theta}(\mathbf{x}) \delta_2)] \\ &= \delta_1^{\top} \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \delta_2 . \end{aligned}$$

$$\mathbf{J}_{\theta}(\mathbf{x}) = \partial f_{\theta}(\mathbf{x}) / \partial \theta \in \mathbb{R}^{D \times P}$$

then we can define an inner product between them as the expected inner product between function outputs

$$\langle f_{\theta}, f_{\theta'} \rangle = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f_{\theta}(\mathbf{x})^{\top} f_{\theta'}(\mathbf{x})].$$



Locally, we can bring this inner product into the parameter space as

$$\begin{aligned} \langle \delta_1, \delta_2 \rangle_{\theta} &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(f_{\theta+\delta_1}(\mathbf{x}) - f_{\theta}(\mathbf{x}))^{\top} (f_{\theta+\delta_2}(\mathbf{x}) - f_{\theta}(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [(\mathbf{J}_{\theta}(\mathbf{x}) \delta_1)^{\top} (\mathbf{J}_{\theta}(\mathbf{x}) \delta_2)] \\ &= \delta_1^{\top} \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \delta_2. \end{aligned}$$

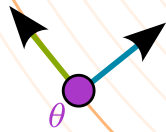
resulting in a metric

$$\mathbf{G}_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

where

$$\mathbf{J}_{\theta}(\mathbf{x}) = \partial f_{\theta}(\mathbf{x}) / \partial \theta \in \mathbb{R}^{D \times P}$$

From zero to hero



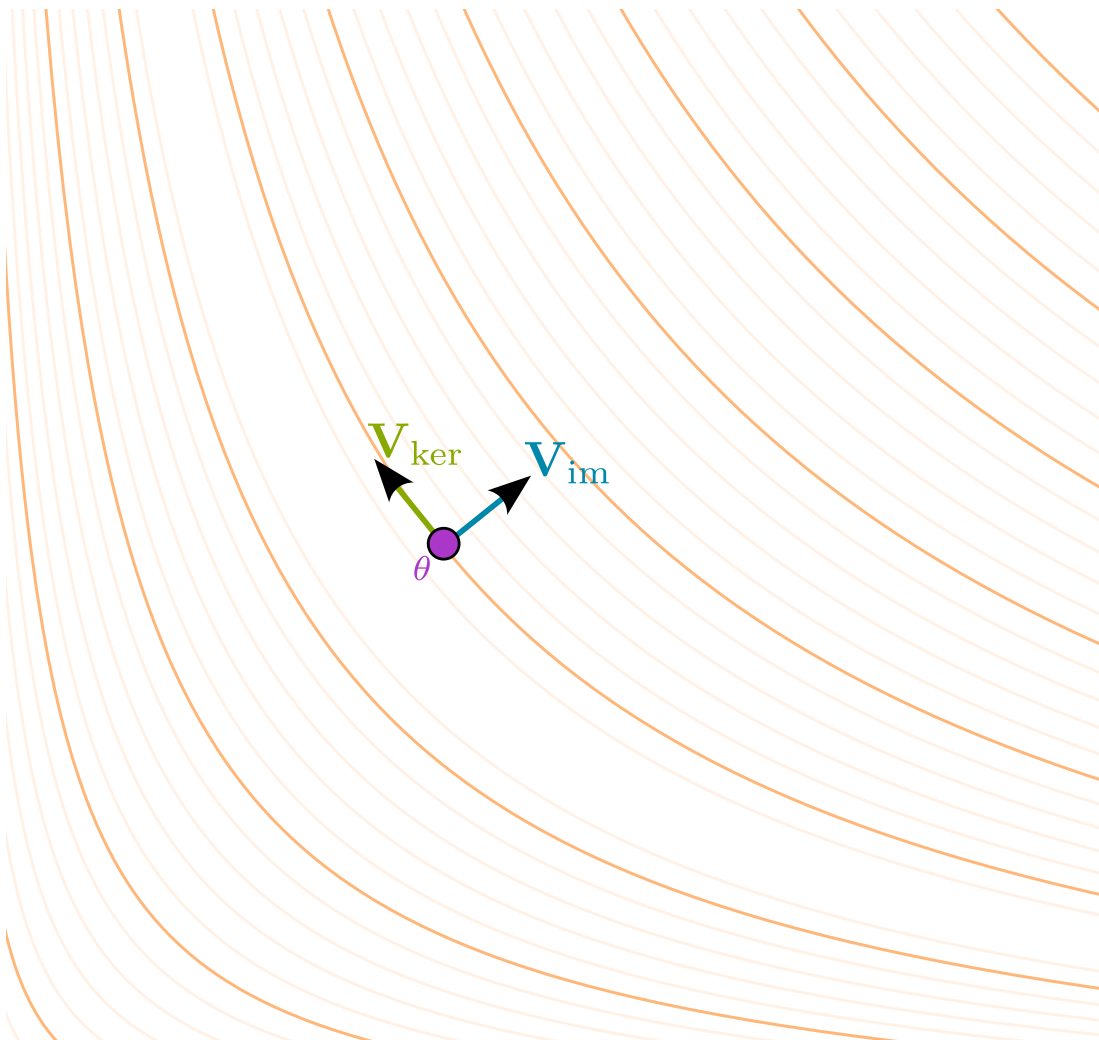
Be aware that distances between distinct parameters can be zero (if they correspond to the same function)

This implies that some eigenvalues of

$$\mathbf{G}_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

are zero with the associated eigenvectors pointing in directions where functions don't change.

From zero to hero



We can explicate this with an eigen-decomposition of the metric

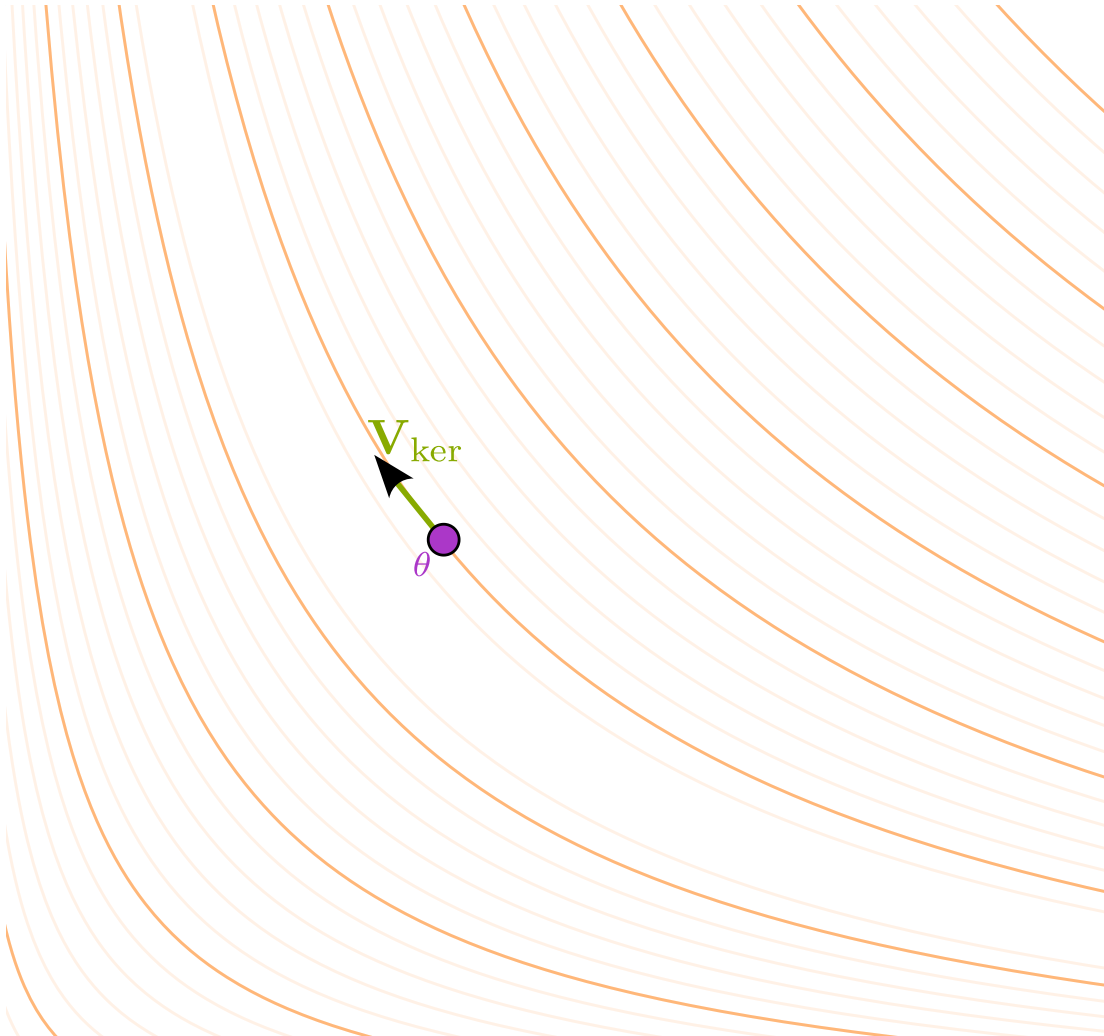
$$\mathbf{G}_\theta = \begin{pmatrix} \mathbf{V}_{\text{im}} & \mathbf{V}_{\text{ker}} \end{pmatrix} \begin{pmatrix} \lambda_{\text{im}} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \mathbf{V}_{\text{im}} & \mathbf{V}_{\text{ker}} \end{pmatrix}^\top$$

where the 'im' and 'ker' subscripts are short for

image: the space spanned by the eigenvectors with nonzero eigenvalues.

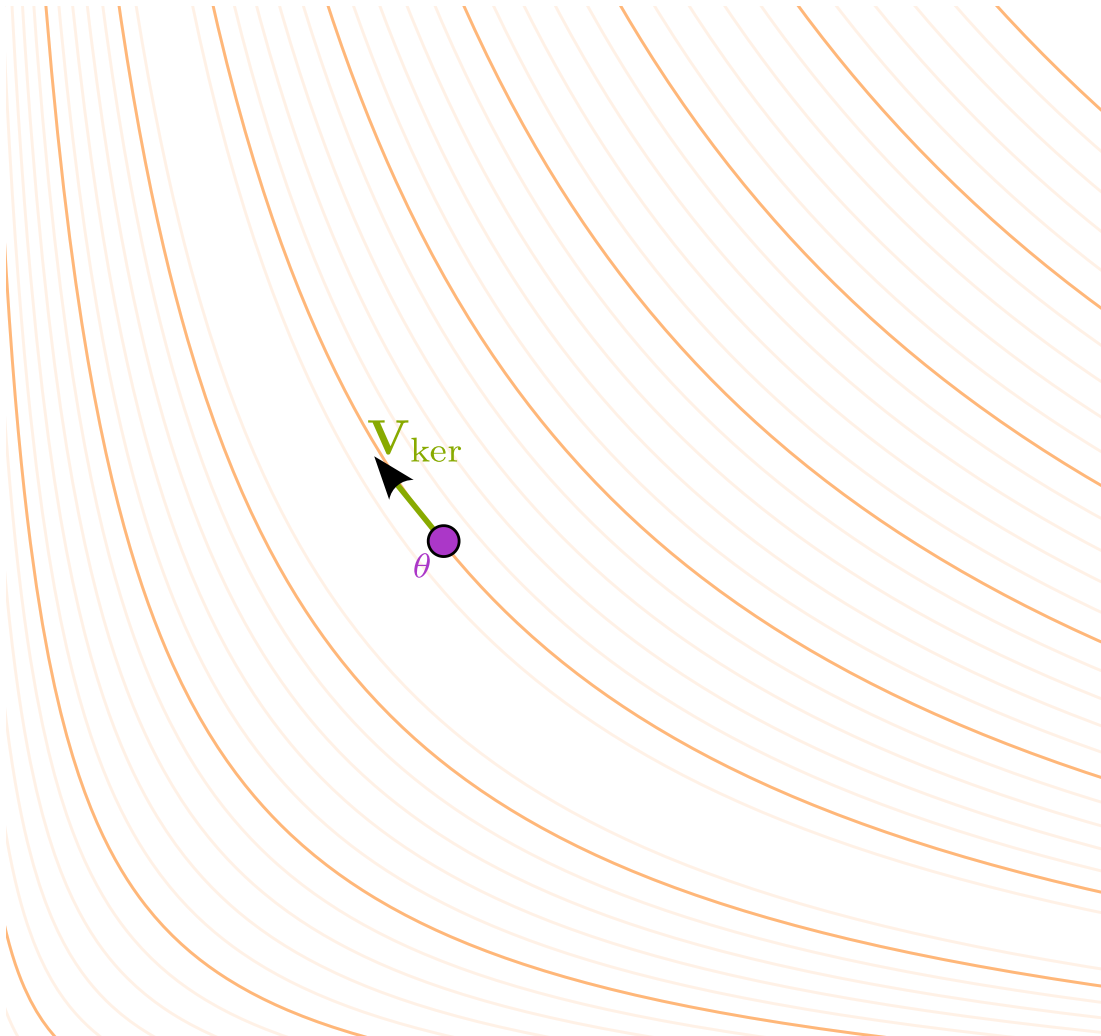
kernel: the space spanned by the eigenvectors with zero eigenvalues.

Kernel manifolds



We call the set of parameters that can be connected with zero-length curves the *kernel manifold*.

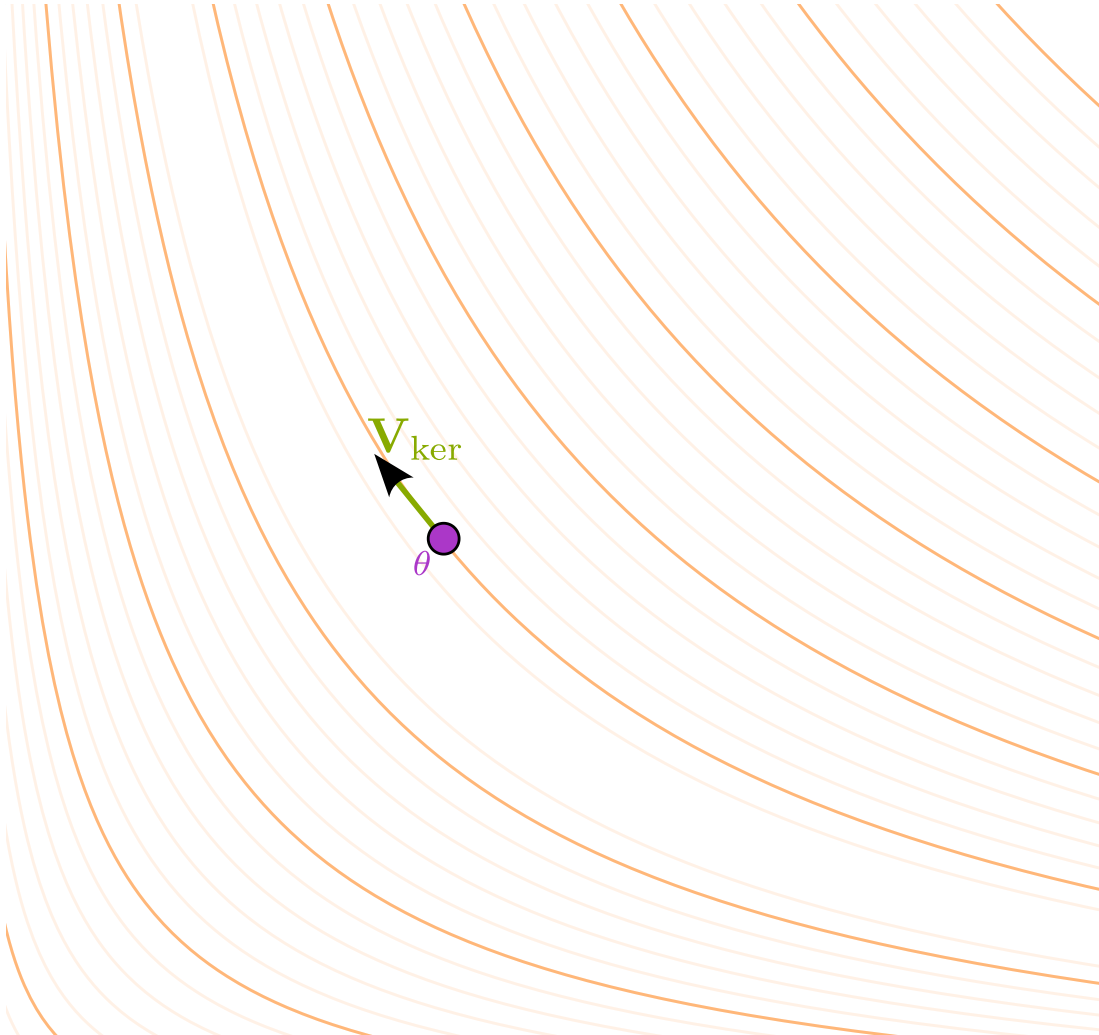
Kernel manifolds



We call the set of parameters that can be connected with zero-length curves the *kernel manifold*.

Specifically, we call $c : [0, 1] \rightarrow \mathbb{R}^P$ an *improper* curve if $\|\dot{c}_t\|_{\mathbf{G}_{c_t}}$ is zero for all t .

Kernel manifolds



We call the set of parameters that can be connected with zero-length curves the *kernel manifold*.

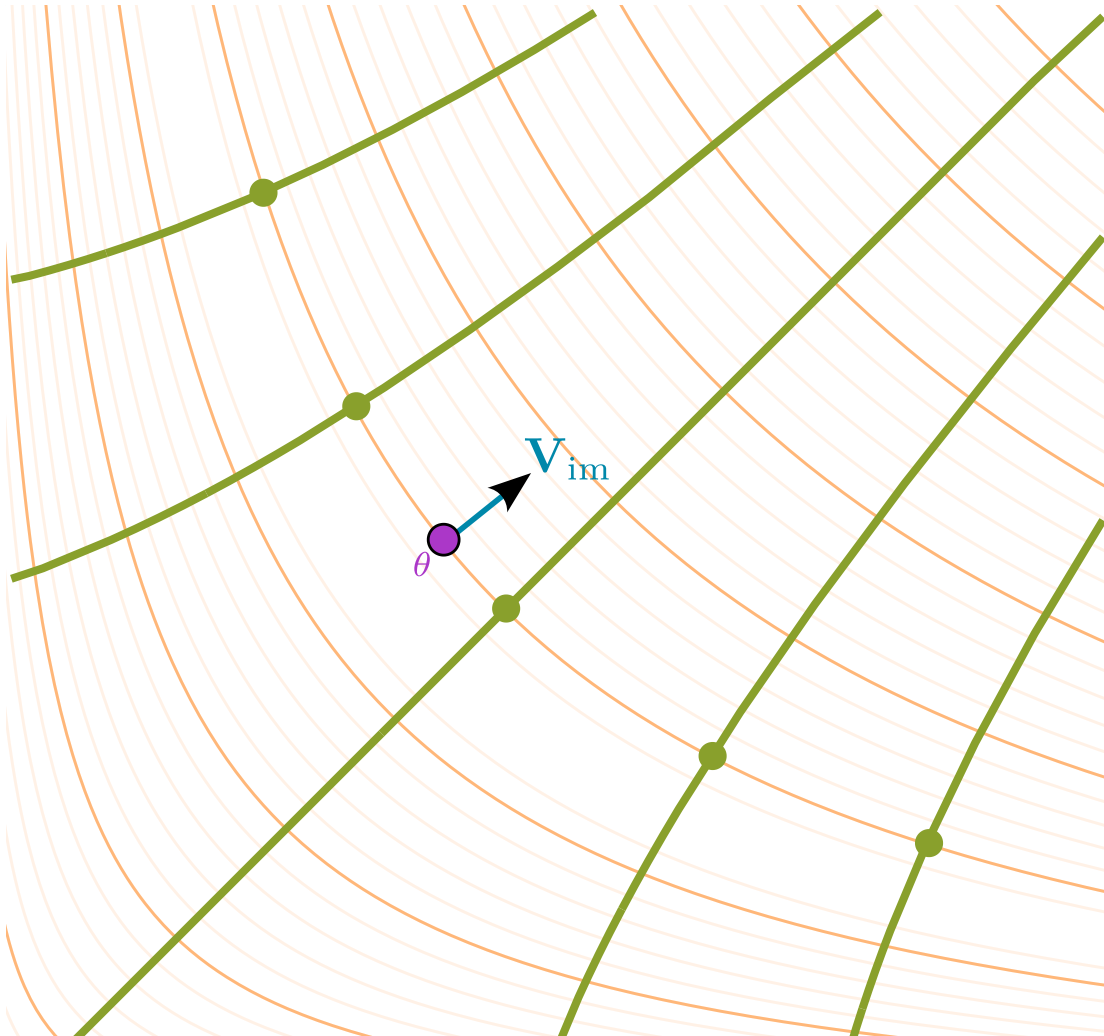
Specifically, we call $c : [0, 1] \rightarrow \mathbb{R}^P$ an *improper* curve if $\|\dot{c}_t\|_{\mathbf{G}_{c_t}}$ is zero for all t .

Then we call

$$\mathcal{M}_{\text{ker}}^\theta = \{\theta' \in \mathbb{R}^P \mid \exists c \text{ an improper curve such that } c_0 = \theta \text{ and } c_1 = \theta'\}.$$

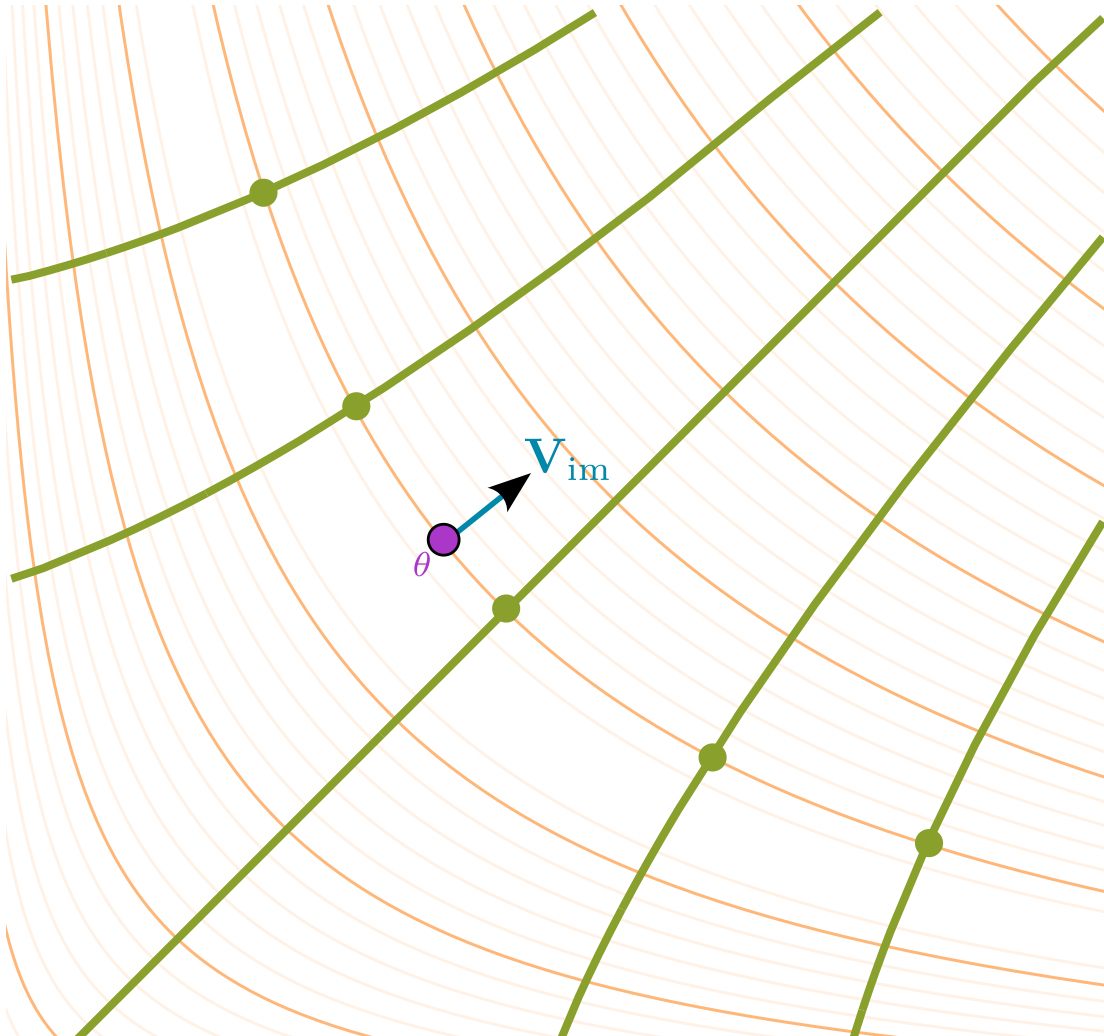
the *kernel manifold* at $\theta \in \mathbb{R}^P$
(the orange curves are such manifolds)

Image manifolds



We call the set of parameters that can be connected with positive-length curves the *image manifold*.

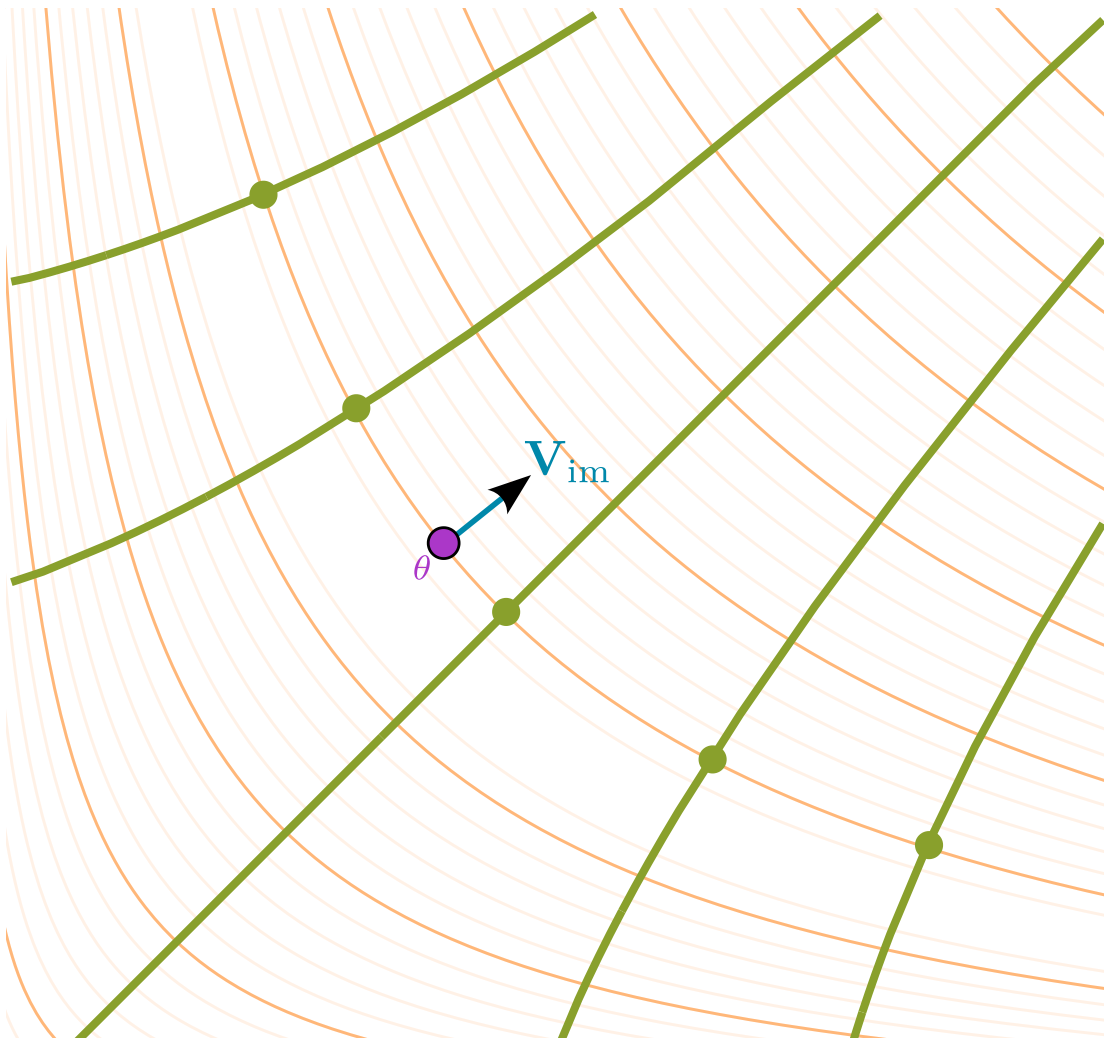
Image manifolds



We call the set of parameters that can be connected with positive-length curves the *image manifold*.

Specifically, we call $c : [0, 1] \rightarrow \mathbb{R}^P$ an *proper* curve if $\|\dot{c}_t\|_{\mathbf{G}_{c_t}}$ is nonzero for all t .

Image manifolds



We call the set of parameters that can be connected with positive-length curves the *image manifold*.

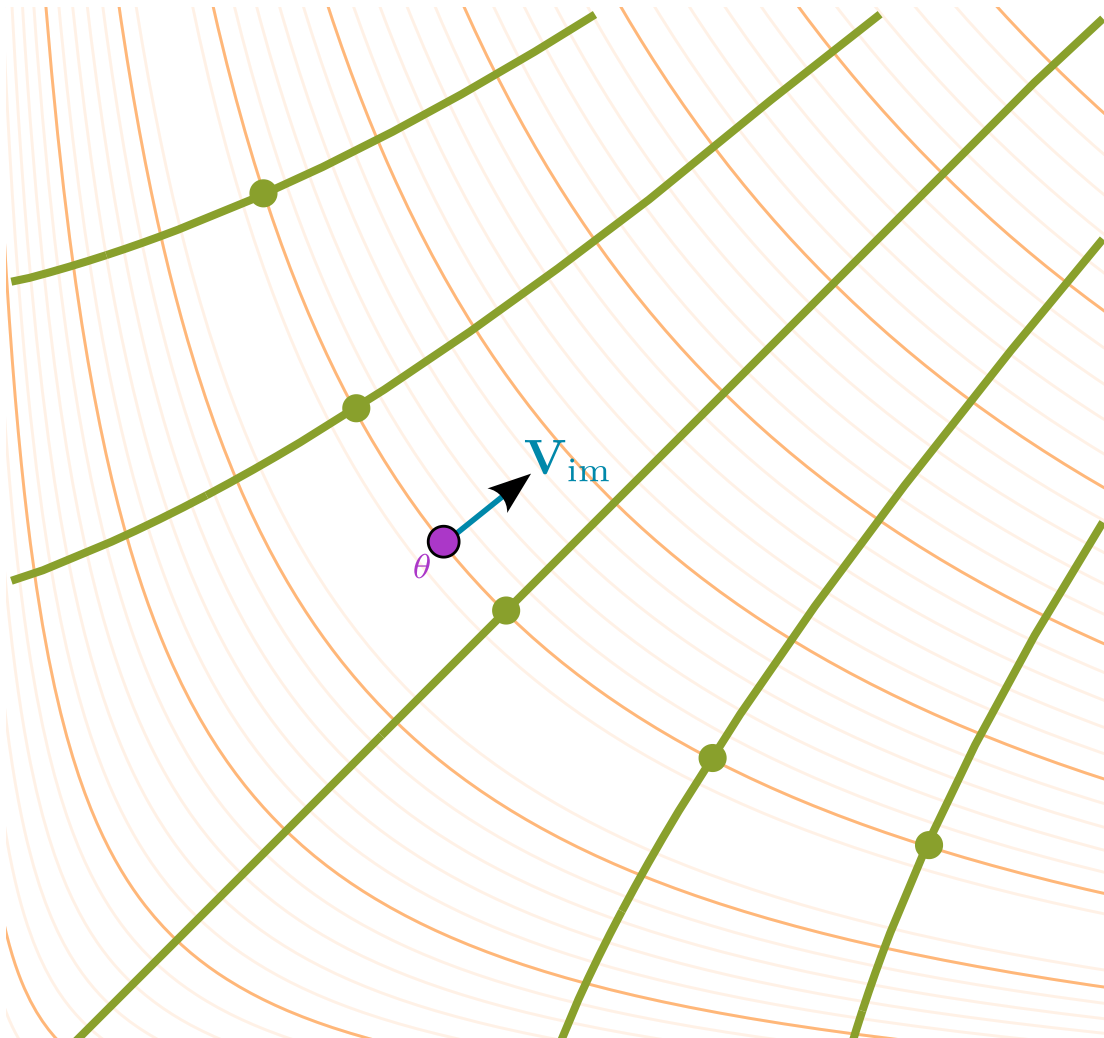
Specifically, we call $c : [0, 1] \rightarrow \mathbb{R}^P$ an *proper curve* if $\|\dot{c}_t\|_{\mathbf{G}_{c_t}}$ is nonzero for all t .

Then we call

$$\mathcal{M}_{\text{im}}^\theta = \{\theta' \in \mathbb{R}^P \mid \exists c \text{ a proper curve such that } c_0 = \theta \text{ and } c_1 = \theta'\}.$$

the *image manifold* at $\theta \in \mathbb{R}^P$
(the green curves are such manifolds)

Image manifolds



We call the set of parameters that can be connected with positive-length curves the *image manifold*.

Specifically, we call $c : [0, 1] \rightarrow \mathbb{R}^P$ an *proper curve* if $\|\dot{c}_t\|_{\mathbf{G}_{c_t}}$ is nonzero for all t .

Then we call

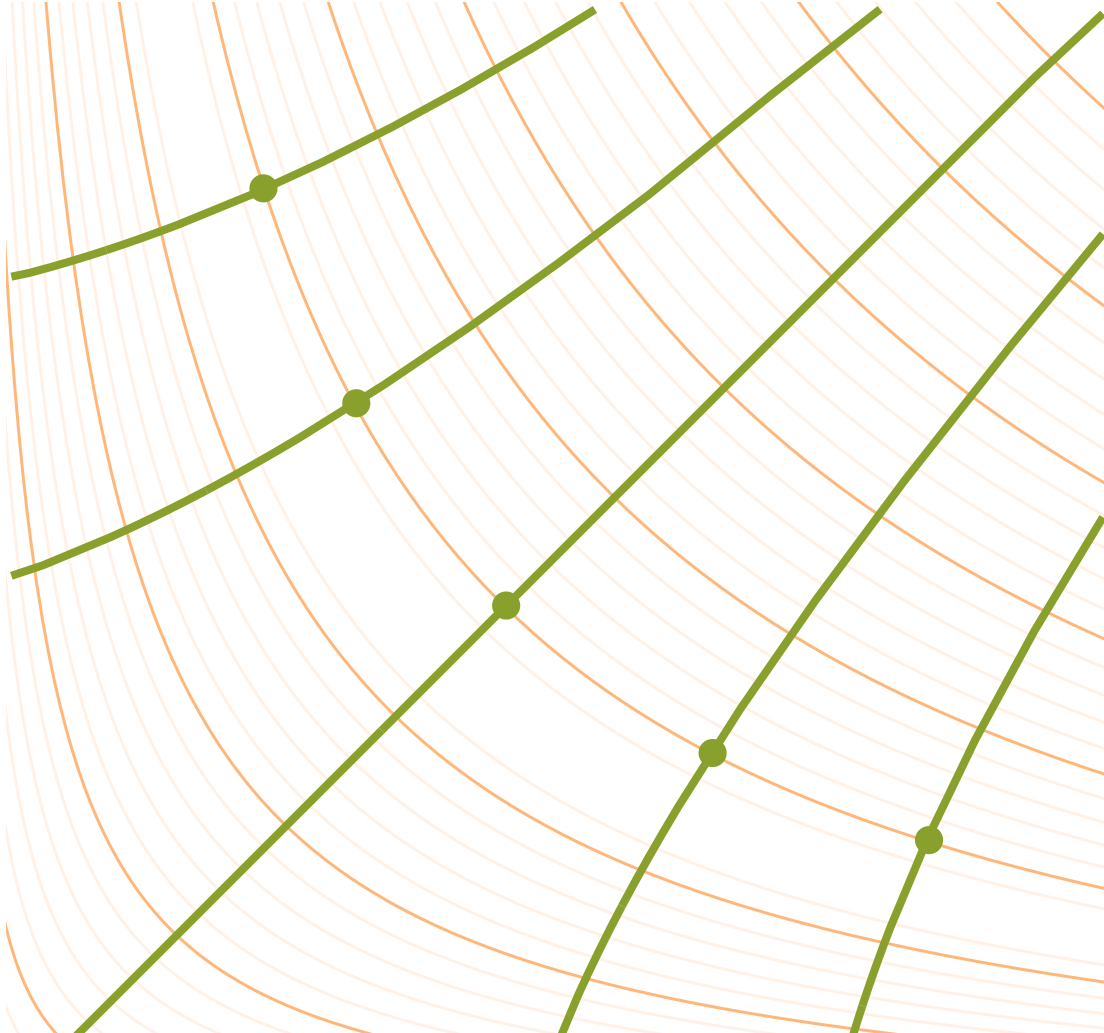
$$\mathcal{M}_{\text{im}}^\theta = \{\theta' \in \mathbb{R}^P \mid \exists c \text{ a proper curve such that } c_0 = \theta \text{ and } c_1 = \theta'\}.$$

the *image manifold* at $\theta \in \mathbb{R}^P$
(the green curves are such manifolds)

The natural metric of an image manifold is

$$\mathbf{G}_\theta^{\text{im}} = \mathbf{V}_{\text{im}} \boldsymbol{\lambda}_{\text{im}} \mathbf{V}_{\text{im}}^\top = \mathbf{G}_\theta.$$

Why is it we care?



This view helps us understand many of the weird behaviors of neural networks.

I'll just show you one example as that is one I care about...

Why is it we care?

These different manifolds are determined by the metric

$$\mathbf{G}_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

Why is it we care?

These different manifolds are determined by the metric

$$\mathbf{G}_\theta = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_\theta(\mathbf{x})^\top \mathbf{J}_\theta(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

In practice, we never have access to the data distribution and instead rely on samples (training data).

$$\bar{\mathbf{G}}_\theta = \frac{1}{S} \sum_{s=1}^S \mathbf{J}_\theta(\mathbf{x}_s)^\top \mathbf{J}_\theta(\mathbf{x}_s)$$

Why is it we care?

These different manifolds are determined by the metric

$$\mathbf{G}_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

In practice, we never have access to the data distribution and instead rely on samples (training data).

$$\bar{\mathbf{G}}_{\theta} = \frac{1}{S} \sum_{s=1}^S \mathbf{J}_{\theta}(\mathbf{x}_s)^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s)$$

Measuring according to this metric (locally) gives

$$\text{dist}_{\bar{\mathbf{G}}_{\theta}}^2(\theta, \theta + \delta) = \delta^{\top} \bar{\mathbf{G}}_{\theta} \delta$$

Why is it we care?

These different manifolds are determined by the metric

$$\mathbf{G}_{\theta} = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_{\theta}(\mathbf{x})^{\top} \mathbf{J}_{\theta}(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

In practice, we never have access to the data distribution and instead rely on samples (training data).

$$\bar{\mathbf{G}}_{\theta} = \frac{1}{S} \sum_{s=1}^S \mathbf{J}_{\theta}(\mathbf{x}_s)^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s)$$

Measuring according to this metric (locally) gives

$$\begin{aligned} \text{dist}_{\bar{\mathbf{G}}_{\theta}}^2(\theta, \theta + \delta) &= \delta^{\top} \bar{\mathbf{G}}_{\theta} \delta \\ &= \frac{1}{S} \sum_{s=1}^S \delta^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s)^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s) \delta \end{aligned}$$

Why is it we care?

These different manifolds are determined by the metric

$$\mathbf{G}_\theta = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\mathbf{J}_\theta(\mathbf{x})^\top \mathbf{J}_\theta(\mathbf{x})] \in \mathbb{R}^{P \times P}$$

In practice, we never have access to the data distribution and instead rely on samples (training data).

$$\bar{\mathbf{G}}_\theta = \frac{1}{S} \sum_{s=1}^S \mathbf{J}_\theta(\mathbf{x}_s)^\top \mathbf{J}_\theta(\mathbf{x}_s)$$

Measuring according to this metric (locally) gives

$$\begin{aligned} \text{dist}_{\bar{\mathbf{G}}_\theta}^2(\theta, \theta + \delta) &= \delta^\top \bar{\mathbf{G}}_\theta \delta \\ &= \frac{1}{S} \sum_{s=1}^S \delta^\top \mathbf{J}_\theta(\mathbf{x}_s)^\top \mathbf{J}_\theta(\mathbf{x}_s) \delta \\ &= \frac{1}{S} \sum_{s=1}^S \|f_{\theta+\delta}(\mathbf{x}_s) - f_\theta(\mathbf{x}_s)\|^2. \end{aligned}$$

and instead rely on samples (training data).

$$\bar{\mathbf{G}}_{\theta} = \frac{1}{S} \sum_{s=1}^S \mathbf{J}_{\theta}(\mathbf{x}_s)^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s)$$

Measuring according to this metric (locally) gives

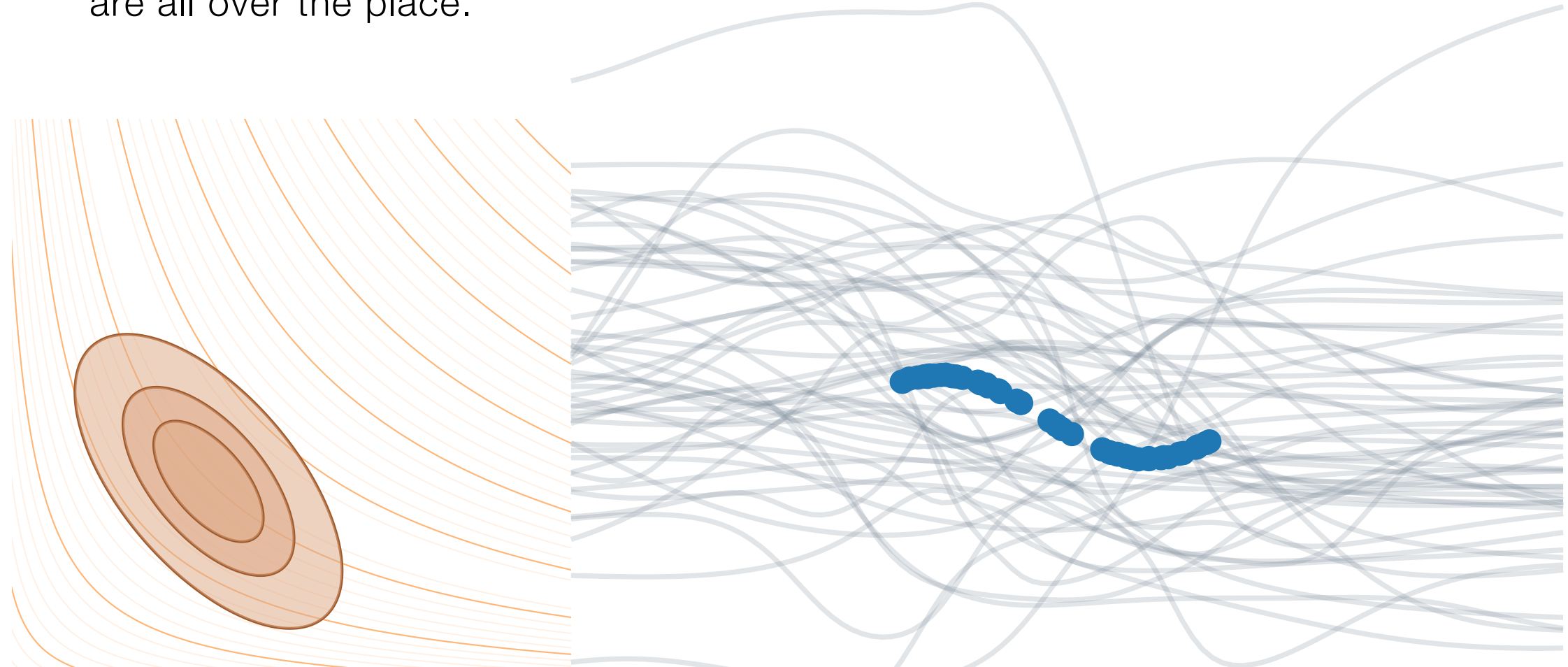
$$\begin{aligned} \text{dist}_{\bar{\mathbf{G}}_{\theta}}^2(\theta, \theta + \delta) &= \delta^{\top} \bar{\mathbf{G}}_{\theta} \delta \\ &= \frac{1}{S} \sum_{s=1}^S \delta^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s)^{\top} \mathbf{J}_{\theta}(\mathbf{x}_s) \delta \\ &= \frac{1}{S} \sum_{s=1}^S \|f_{\theta+\delta}(\mathbf{x}_s) - f_{\theta}(\mathbf{x}_s)\|^2. \end{aligned}$$

We measure changes in
predictions on the training data

*(two zero-distance functions
according to the sample metric)*

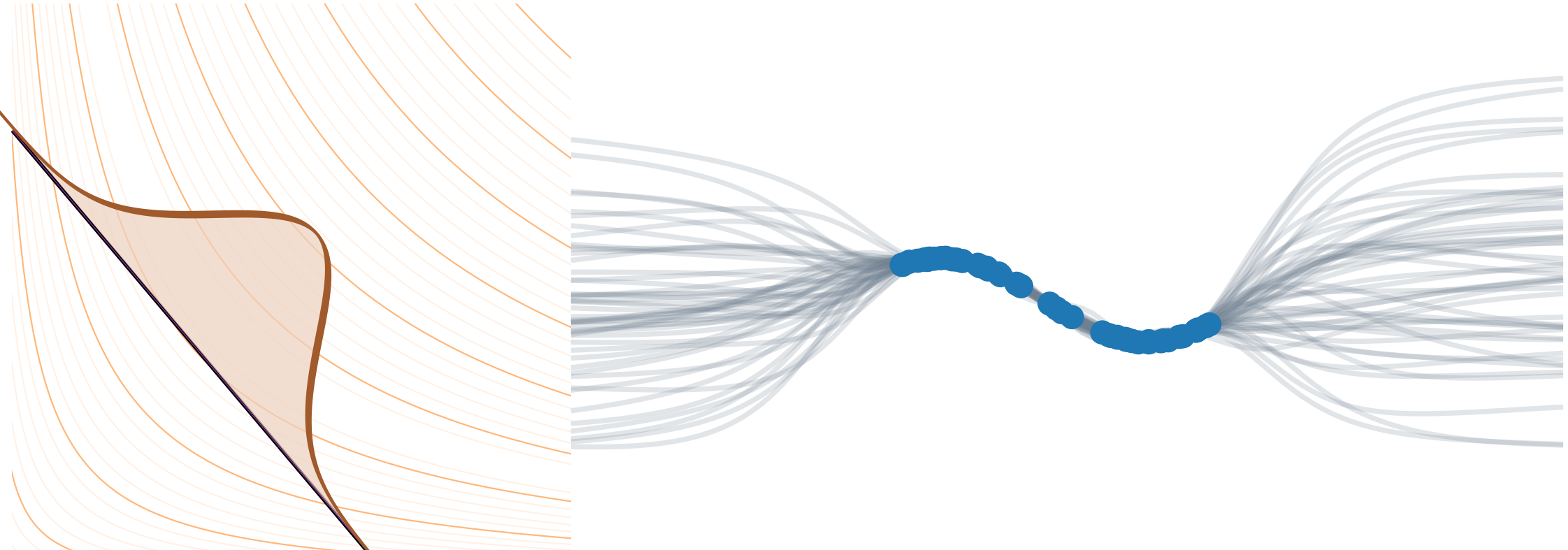
Why is it we care?

If you draw Gaussian samples in parameter space, you get functions that are all over the place.



Why is it we care?

Projecting to the kernel gives samples that interpolate the data.
Wildly enough, this is the current state-of-the-art in uncertainty quantification.



Exercises

You should complete exercises in the exercise sheet

Exam relevant

Exercise 7.1, 7.2, 7.3, 7.4, 7.5

The first two concern noisy manifolds, while the last three are about parameter manifolds (the last one is easy if you have done the fourth).

Project relevant

Mini-project 2

Fit ensemble VAEs and compute pullback geodesics numerically.